

Università degli Studi di Napoli
Federico II

A.A. 2025-2026

# Reti e Programmazione Distribuita

# Indice


0 Note sul corso 3

I DNS 4

1 DNS: Domain Name System 5
- DNS: più di una «rubrica telefonica» 5
- DNS: i nomi 5
- Domini, zone e delega 5
  - Dominio vs Zona 5
- DNS: tipi di record 6
  - Principali tipi di record DNS 6
- DNS: gli attori 6
- Query ricorsive vs iterative 7
- Interazioni DNS: query e risposte 7
- Risposte DNS 7
  - Risposte NOERROR 8
- Esempio di risoluzione: cold-cache 8
  - La cache nella risoluzione DNS 8
- Pratica 8

II HTTP 10

2 Lezione 2: HTTP: The Hypertext Transfer Protocol 11
- Il Contesto: Web Technologies 11
  - Ipertesti 11
- HTTP: Hypertext Transfer Protocol 11
  - Caratteristiche e Ruolo 11
  - URL: Uniform Resource Locator 11
  - Intestazioni delle Richieste 12
- Risposte HTTP 12
  - Struttura di una Risposta 12
  - Codici di Stato HTTP 12

L'Intestazione Host 12
Caratteristica Statelessness 13
Server HTTP 13
Definizione e Ruoli 13
Modalità di Elaborazione delle Richieste 13
Server HTTP Statici 13
Risoluzione DNS per Siti Web Locali 14

3 Lezione 3 : HTTP Rappresentazioni, Efficienza e Sicurezza 15
Content Negotiation 15
Principi fondamentali 15
Pesi e fallback 15
Header di negoziazione comuni 16
Caching 16
Benefici 16
Problematiche critiche della cache 16
Livelli, cache private e condivise 16
Il dilemma della consistenza 17
Tre decisioni fondamentali 17
L'header Cache-Control 17
Freschezza 18
Validatori: ETag e Last-Modified 18
Caching e Content Negotiation 19
HTTPS e Sicurezza della Comunicazione 20
HTTPS e TLS 20
Proprietà e limiti 20
HTTP/2: Multiplexing e Efficienza 20
Limitazioni di HTTP/1.1 con richieste parallele 21
Multiplexing in HTTP/2 21
HTTP/3 e QUIC 22
Caratteristiche di QUIC 22

III HTML 23

4 Lezione 4 HTML: HyperText Markup Language 24
Web e HTML 24
Struttura del documento 24
Commenti 24
Elementi fondamentali 25
Titoli e paragrafi 25
Semantica a livello di testo 25
Ancore e URL 25
Tabelle 26
Liste 26
Riferimenti di carattere 26

Immagini 26
Form 27
Controlli di input 27
Invio (submission) 27
Organizzazione del contenuto 28
Errori HTML e recupero dei browser 29
DevTools 30

IV CSS 31

5 Lezione 05 CSS: Cascading Style Sheets 32
Ruolo del CSS nel web 32
Stili di default del browser (user agent styles) 32
Includere stili in una pagina 32
Selettori 33
Selettori semplici 33
Selettori composti (compound) 33
Combinatori 34
Pseudo-classi 34
Pseudo-elementi 35
La cascata (the cascade) 35
Origine e importanza 36
Layers 36
Specificità 37
Posizione e ordine di apparizione 38
Ereditarietà (inheritance) 38

6 Lezione 06 CSS: Layouts, Responsive Design 39
Unità di misura (sizing units) 39
Panoramica 39
Unità assolute 39
Percentuali 39
Unità relative al font 39
Unità relative al viewport 40
Il box model 40
Layout 41
Flow layout (normal flow) 41
La proprietà display 41
Floats 41
Positioning 41
Layout moderni: Flexbox e Grid 42
Flexbox: flex container 42
Flexbox: flex items 42
Flexbox: gestione dello spazio libero 43
Grid 43

Media queries 44
Tipi di output 44
Media features 44
Responsive design 45
Fixed-width layouts 45
Liquid (fluid) layouts 45
Siti separati (mobile) 45
Adaptive layouts 45
Responsive layouts 46
Viewport meta tag 46

## 7 Lezione 07: Modern, Adaptive CSS 48

Pseudo-classi funzionali 48
`:is()` (matches-any) 48
`:where()` 48
`:not()` 48
`:has()` 49
Specificità delle pseudo-classi funzionali 49
Custom properties (variabili CSS) 49
Ereditarietà 49
Fallback 50
At-rule `@property` 50
Valori calcolati 50
`calc()` 50
`min()` e `max()` 50
`clamp()` 50
Unità viewport su dispositivi mobili 51
Pattern di layout moderni 51
`gap` 51
Shorthand `flex` 51
Container queries 51
Candidati immagine responsive 52
`srcset` 52
`<picture>` e art direction 52
Media query e preferenze utente 53
Query `@supports` 53
Nesting nativo del CSS 53
Selettore di nesting: `&` 54

# V JavaScript 55

## 8 Lezione 8 JavaScript 56

Il linguaggio 56
Includere JavaScript in una pagina 56

Strict mode 56
Sintassi di base 57
Operatori 57
Controllo di flusso 57
Variabili e scope 57
Ciclo di vita 58
Dichiarazione 58
Hoisting 58
`var` e dichiarazione implicita (pre-ECMAScript 6) 59
Scope 59
Funzioni 59
Scope interno e visibilità 59
Function expressions 60
Funzioni annidate e closure 60
Oggetti 60
Proprietà 60
Riferimenti 61
Cloning 61
Metodi 61
`this` 61
Costruttori 62
Optional chaining 62
Configurazione delle proprietà 62
Getter e setter 62

9 Lezione 9 JavaScript 64
Prototipi ed ereditarietà 64
Il prototype 64
Metodi: due alternative 65
Strutture dati 65
Array 65
Destructuring 66
Iterabili e Symbol.iterator 66
Map 67
Set 67
Classi 68
La sintassi class 68
Ereditarietà: extends 68
Gestione degli errori 69
Errori e try/catch/finally 69
instanceof 70
throw 70
Moduli 71
Modularità 71
export e import 71

10 Lezione 11 --- JavaScript in a Browser Environment 72
L'ambiente browser 72
Host environment 72

window 72
DOM 73
BOM 73
Il DOM 73
Struttura ad albero 73
Tipi di nodo 74
Trovare elementi 74
Navigare nel DOM 75
Navigazione solo elementi 75
Gerarchia delle classi dei nodi 76
Proprietà dei nodi 76
Manipolare il DOM 77
Creare nuovi elementi 78
Metodi di inserimento 78
Interagire con l'utente 79
Eventi 79
Definizione e classificazione 79
Handler 80
`this` negli handler 80
L'oggetto event 80
Event bubbling 81
Event delegation 81
Fermare il bubbling 81
Generare eventi (dispatching) 82
Capturing 82
Ordine di esecuzione degli script 83

## 11 Lezione 11 JavaScript asincronismo e richieste di rete 85

Storage dei dati nel browser 85
Cookie 85
localStorage 85
sessionStorage 85
IndexedDB 86
Sincronismo e asincronismo 86
Codice sincrono 86
Codice asincrono 86
Callback 86
Limiti delle callback 87
Promise 87
Produttore e consumatore 87
Creazione di una Promise 87
Ciclo di vita 88
then 88
catch 88
finally 88
Chaining 89
Metodi statici dell'API Promise 89
async e await 90
async 90
await 90

Richieste di rete 91
XMLHttpRequest 91
fetch 91
Lavorare con la risposta 91
Corpo della risposta 92

VI SERVER SIDE 93

12 LEZIONE 12 WEB STATICO E DINAMICO 94
Richiami su Promise e Fetch 94
Promise 94
Fetch API 94
Il web statico 95
App web statiche 95
Limitazioni del web statico 95
Programmazione server-side 96
Generazione on-the-fly 96
CGI 96
Definizione 96
Flusso di una richiesta 96
Input e output del programma 97
Primo script CGI 97
Server-side scripting 97
Limiti dell'approccio CGI 97
Linguaggi di scripting server-side 98
PHP 98

13 LEZIONE 13 NODE.JS 100
JavaScript in ambiente server 100
Gestione multi-threaded delle richieste 100
La filosofia di Node.js 101
Il single-threaded event loop 101
Installazione 101
Node.js vs browser 102
npm 102
Il file package.json 102
Installare le dipendenze 103
Distribuire un pacchetto 103
Una prima applicazione web 103
Debugging in VS Code 104
Auto Attach 104
Azioni di debug 104
Live reloading 105

Capitolo 0

# Note sul corso

I seguenti appounti sono stati presi durante le lezioni del corso di Tecnologie Web.
**Nota:**

# Parte I

# DNS

Capitolo 1

# DNS: Domain Name System

## DNS: più di una «rubrica telefonica»

- A volte il DNS viene descritto come la «rubrica telefonica» di Internet
- In realtà il DNS è un database **distribuito e gerarchico**

## DNS: i nomi

- I nomi DNS sono una sequenza di **etichette** (label)
  - Convenzionalmente stampate separate da punti
  - La gerarchia si legge da destra verso sinistra; la specificità aumenta da destra verso sinistra
  - Le etichette non distinguono maiuscole/minuscole (case insensitive)
- Si consideri: `informatica.dieti.unina.it.`
  - L'etichetta vuota dopo il punto finale è la «radice» (root)
  - «root» → `it` → `unina` → `dieti` → `informatica`
- I nomi DNS sono talvolta chiamati informalmente *hostname*
  - Possono non corrispondere a un host, ma a un servizio, a un punto di delega, a una policy di instradamento della posta, o semplicemente a un nodo del database DNS distribuito
  - Inoltre, lo stesso host può avere nomi DNS diversi

## Domini, zone e delega

- Un **dominio** è una parte dello spazio dei nomi (namespace)
  - Un nodo e il sottoalbero che vi è radicato
- Tutti i domini di primo livello e molti domini di secondo livello e inferiori sono suddivisi, tramite **delega**, in unità più piccole e gestibili
- Queste unità più piccole si chiamano **zone**
- Ogni zona è servita da uno o più **name server autoritativi**
  - I name server memorizzano e forniscono le informazioni DNS per le zone
  - Possono esistere più name server autoritativi per ridondanza
  - L'autorità di un name server termina in corrispondenza di zone figlie delegate
  - Un name server può essere autoritativo per molte zone

#### Dominio vs Zona

Dominio e zona sono concetti diversi:

- Il **dominio** è un concetto **strutturale** (un sottoalbero del namespace)

- La **zona** è un concetto **amministrativo** (la parte del namespace per la quale un'autorità pubblica dati)
- È la delega a renderli diversi!

Se **unina.it.** delega **dieti.unina.it.** al DIETI:

- **dieti.unina.it.** smette di far parte del dominio **unina.it.**? **No.** Fa ancora parte del sottoalbero radicato in `unina.it.`
- **dieti.unina.it.** smette di far parte della zona **unina.it.**? **Sì.** L'autorità è stata delegata!

> I domini descrivono l'albero. Le zone descrivono la responsabilità amministrativa su parti dell'albero.

# DNS: tipi di record

Il DNS memorizza le informazioni come **Resource Record** (RR):

| Owner name | TTL | Class | Type | Value |
|---|---|---|---|---|
| informatica.dieti.unina.it. | 21600 | IN | A | 143.225.97.81 |

- **Owner name**: il nome DNS descritto dal record
- **TTL**: time-to-live, ovvero per quanto tempo la risposta può essere conservata in cache (in secondi)
- **Class**: normalmente «IN», ovvero Internet
- **Type**: il tipo di informazione contenuta nel record
- **Value**: il dato associato al nome

**Principali tipi di record DNS**

| Tipo | Scopo |
|---|---|
| A | Da nome a indirizzo IPv4 |
| AAAA | Da nome a indirizzo IPv6 |
| CNAME | Alias verso un nome canonico |
| NS | Server autoritativi di una zona |
| SOA | Identifica una zona |
| MX | Server di posta per un dominio |

# DNS: gli attori

Come fanno i browser (o i client in generale) a passare da un nome DNS a un indirizzo IP? Nel processo sono coinvolti diversi attori:

- Il **client** chiede a un'interfaccia locale di risoluzione (**stub resolver**), gestita a livello di sistema operativo, un record di un certo tipo (es. A) per un dato nome

- Lo stub resolver delega la risoluzione a un **resolver ricorsivo**
  - Tipicamente gestito da ISP o provider pubblici (Google – 8.8.8.8, Cloudflare – 1.1.1.1, …)
- Il resolver ricorsivo naviga la **gerarchia autoritativa** cercando il record
  - Ciò può richiedere il «dialogo» con più server DNS
- Il resolver ricorsivo restituisce il record oppure un errore

## Query ricorsive vs iterative

- **Query ricorsive**: «Trova la risposta definitiva per me»
  - Dal client → al resolver ricorsivo
  - Il resolver svolge tutto il lavoro
  - Nella ricorsione, il server interpellato è responsabile di ottenere il risultato
- **Query iterative**: «Dammi la migliore informazione che hai»
  - Dal resolver ricorsivo → ai server DNS
  - Ogni server può restituire una risposta o un referral
  - Nell'iterazione, il resolver segue i referral e decide quale sia la query successiva

## Interazioni DNS: query e risposte

Un'interazione DNS consiste normalmente di due messaggi:

- **Question** (domanda)
  - Es. `informatica.dieti.unina.it. IN A`
- **Response** (risposta)
  - *Header*: stato e flag
  - *Question*: riportata dalla query (echo)
  - *Answer*: record che rispondono alla domanda
  - *Authority*: dati autoritativi o referral
  - *Additional*: record di supporto utili

Le domande contengono tre campi principali:

- **NAME** (il nome richiesto)
- **CLASS** (normalmente IN per Internet — talvolta omessa)
- **TYPE** (il tipo di record richiesto)

## Risposte DNS

Gli header delle risposte DNS iniziano con un **codice di stato**. I possibili codici includono:

| Stato | Significato |
|---|---|
| **NOERROR** | Il server ha elaborato la query con successo |
| **NXDOMAIN** | Il nome richiesto non esiste |
| **SERVFAIL** | Il server non è riuscito a completare la query |

| Stato | Significato |
|---|---|
| **REFUSED** | Il server ha rifiutato di eseguire l'operazione richiesta |
| **FORMERR** | Il messaggio di query era malformato |

**Risposte NOERROR**

- **NOERROR** non significa necessariamente che il record sia stato trovato
- Significa che il server DNS ha capito ed elaborato la query
- Sono possibili scenari diversi:
  - **Risposta positiva.** Il blocco Answer contiene il record richiesto
  - **Referral.** Il blocco Authority contiene record **NS** di una zona che potrebbe sapere di più
  - **NODATA.** La risposta può confermare che il nome esiste, ma non esiste alcun record del tipo richiesto (blocco Answer vuoto)

# Esempio di risoluzione: cold-cache

Il client vuole ottenere un record A per `squids.unina.it.`:

1. Il client chiede allo stub resolver
2. Lo stub resolver invia una query DNS al resolver ricorsivo
3. Il resolver ricorsivo interroga la root → riceve un referral verso `.it`
4. Il resolver ricorsivo interroga `.it` → riceve un referral verso `unina.it`
5. Il resolver ricorsivo interroga `unina.it` → riceve il record A
6. Il resolver ricorsivo risponde allo stub resolver

**La cache nella risoluzione DNS**

- Nello scenario precedente il resolver ricorsivo ha navigato l'intera gerarchia
- Nel mondo reale, i sistemi memorizzano i dati in una cache per efficienza
  - Questo vale per browser, stub resolver, resolver ricorsivi, …
- Se un record è già in cache (e non è scaduto), non serve alcuna navigazione!
- Lo scenario in cui nessun dato in cache è disponibile durante la risoluzione è lo scenario peggiore
  - Chiamato anche **risoluzione cold-cache**
- Il **TTL** determina per quanto tempo un record in cache può essere usato

# Pratica

**Lab**

- Con **dig**, analizziamo query e risposte DNS; l'opzione **+trace** mostra la risoluzione gerarchica completa, senza cache, dai root server fino ai server autoritativi:

```Bash
luigi@XPS-9520:/$ dig @8.8.8.8 squids.unina.it. +trace
```

- Ogni livello rimanda ai name server del livello successivo: il DNS di Google elenca i root server, il root server selezionato rimanda ai name server del TLD `.it`, questi a quelli di `unina.it`, e infine il name server autoritativo (`dscna2.unina.it`) restituisce il record A:

```Bash
.                   87203   IN      NS      h.root-servers.net.
it.                 172800  IN      NS      m.dns.it.
unina.it.           3600    IN      NS      dscna2.unina.it.
squids.unina.it.    1800    IN      A       143.225.131.206
```

- Se il nome **non esiste** (es. `webtechnologies.unina.it.`), la risposta ha status **NXDOMAIN** e il browser mostra «Server Not Found»:

```Bash
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN
;webtechnologies.unina.it.      IN      A
```

- Se invece il nome **esiste ma non ha record A**, la risposta è **NODATA** (status NOERROR, nessuna sezione ANSWER) — è il caso di `docenti.unina.it` rispetto a `www.docenti.unina.it`, che sono **nomi diversi**: solo il primo ha un record A, quindi il secondo restituisce «Server Not Found»:

```Bash
$ dig www.docenti.unina.it A     → NOERROR, A 143.225.212.48
$ dig docenti.unina.it A         → NOERROR, nessuna ANSWER SECTION
```

> *Nota: il DNS è usato anche per ottenere l'IP (record A) del NS selezionato, se la risposta intermedia non lo contiene già nella sezione additional!*

# Parte II

# HTTP

Capitolo 2

# Lezione 2: HTTP: The Hypertext Transfer Protocol

## Il Contesto: Web Technologies

**Internet** è una rete globale di computer interconnessi che condividono informazioni tramite i protocolli Internet; le sue origini risalgono ad **ARPANET** (1969). La **World Wide Web** (WWW, o semplicemente "il Web") non coincide con Internet: ne è un sottoinsieme. Fu inventata da Sir Tim Berners-Lee all'inizio degli anni '90 come sistema di **documenti ipertestuali** interconnessi tramite **hyperlink**, e i suoi componenti fondamentali sono **HTTP** e **HTML**.

### Ipertesti

I documenti tradizionali sono semplici sequenze di caratteri. Un **ipertesto**, invece, è un documento che contiene anche collegamenti ad altri contenuti, come altri ipertesti, documenti o media.

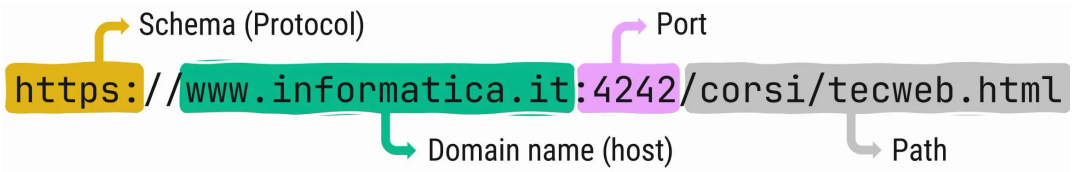
## HTTP: Hypertext Transfer Protocol

### Caratteristiche e Ruolo

**HTTP** è un **protocollo applicativo** costruito sopra **TCP/IP** e rappresenta la fondazione e la spina dorsale della World Wide Web. Sviluppato originariamente per la trasmissione di ipertesti, è oggi utilizzato anche per la trasmissione di altre tipologie di risorse. Nel modello HTTP, il **client invia una richiesta** al server per interagire con una risorsa, e il **server risponde** alla richiesta. Le risorse sono identificate da **URL (Uniform Resource Locator)**.

### URL: Uniform Resource Locator

Un URL completo ha la forma:



### Intestazioni delle Richieste

Le intestazioni HTTP sono meccanismi per passare informazioni aggiuntive nelle richieste e nelle risposte. Un'intestazione è composta da un nome (case-insensitive) seguito da un due-punti e dal valore: `HEADER_NAME: value`.

# Risposte HTTP

### Struttura di una Risposta

Una risposta HTTP è composta da:

- **Linea di Stato**: contiene la versione del protocollo, il codice di stato, e un messaggio descrittivo.
- **Intestazioni (Headers)**: metadati che descrivono la risposta.
- **Linea Vuota**: separa le intestazioni dal corpo della risposta.
- **Corpo della Risposta (facoltativo)**: contiene la risorsa richiesta o ulteriori informazioni.

**Example**

HTTP

```http
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 153
```

### Codici di Stato HTTP

I codici di stato indicano se una richiesta è stata completata con successo. Sono organizzati in cinque classi:

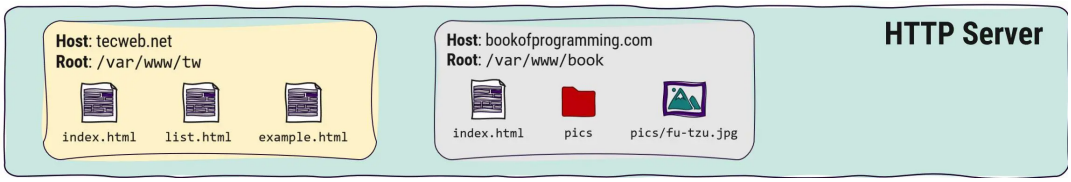
- **1xx (Informazionale)**: Lo scambio richiesta-risposta è in corso.
- **2xx (Successo)**: La richiesta è stata elaborata con successo.
- **3xx (Reindirizzamento)**: Il client deve intraprendere un'azione aggiuntiva, ad esempio un reindirizzamento.
- **4xx (Errore della Richiesta)**: La richiesta non può essere elaborata come presentata (errore nel client).
- **5xx (Errore del Server)**: Il server, o un componente intermedio come un database, ha fallito.

### L'Intestazione Host

Una questione che sorge naturalmente è: perché le richieste HTTP includono un'intestazione **Host** quando il nome di dominio è già stato risolto in indirizzo IP dal DNS?

In realtà **più nomi di host possono risolvere allo stesso IP** — ad esempio `www`, `informatica`, `biblioteca` ed `erasmus.dieti.unina.it` puntano tutti a 143.225.97.81. Poiché un server HTTP può essere configurato per gestire più **virtual host**, ciascuno con la propria document root, è proprio l'header **Host** a permettergli di selezionare il virtual host corretto, poiché il

server non conosce automaticamente quale dominio il client ha digitato nel browser; conosce solo l'indirizzo IP verso cui la connessione è stata stabilita.




# Caratteristica Statelessness

**HTTP è un protocollo stateless**, il che significa che ogni richiesta è indipendente dalle precedenti e il server non mantiene informazioni sui progressi delle richieste precedenti di un client.

Questa semplicità è un vantaggio architetturale, ma per implementare conversazioni "statefull" (con memória del contesto) è necessario ricorrere a meccanismi specifici, come i **cookie**.

# Server HTTP

### Definizione e Ruoli

Un **server HTTP** è un software in esecuzione su una macchina host che: - è in ascolto su una porta specifica (ad esempio 80 per HTTP, 443 per HTTPS); - riceve richieste HTTP; - interpreta il metodo, la destinazione e le intestazioni della richiesta; - produce una risposta HTTP.

Il termine "server" può riferirsi a più entità: - il **ruolo** del server come partecipante che risponde alle richieste; - il **software** server che ascolta le richieste HTTP e risponde; - il **processo** come istanza esecutiva del software; - la **macchina** su cui il software è in esecuzione.

### Modalità di Elaborazione delle Richieste

Quando un server gestisce una richiesta, può costruire la risposta in modi diversi. - Può recuperare un file dal proprio file system (servendo **file statici**) e includerne il contenuto nel corpo della risposta. - Può **eseguire codice per generare una risposta dinamicamente**, il che può coinvolgere interrogazioni di database e interazioni con altri sistemi, con complessità arbitraria.

Tra i server HTTP statici più noti figurano **nginx** e **Apache httpd**.

### Server HTTP Statici

I server statici mappano le risorse richieste a file nel filesystem, secondo una configurazione decisa dal server. Tipicamente si servono i file a partire da una directory specifica, la **document root**: se questa è `/var/www/`, la richiesta `GET /webtech/hello.txt` restituisce

`/var/www/webtech/hello.txt`. La root e molte altre impostazioni si definiscono nei file di configurazione dedicati del server.

## Risoluzione DNS per Siti Web Locali

Il suffisso `.local` non è un **Top-Level Domain (TLD)** che può essere acquistato pubblicamente. Per fare in modo che i nomi di dominio vengano risolti all'indirizzo IP corretto (127.0.0.1), è necessario ricorrere a un meccanismo locale. Nei sistemi operativi moderni, è possibile utilizzare il **file hosts** per ottenere questo risultato.

I **risolutori DNS stub** (le componenti locali del sistema di risoluzione dei nomi) consultano questo file prima di inviare la richiesta a un risolutore ricorsivo remoto. Aggiungendo al file hosts una voce come `127.0.0.1 book-of-programming.local`, le richieste DNS per questo dominio verranno automaticamente risolte a 127.0.0.1 (localhost).

Capitolo 3

# Lezione 3 : HTTP Rappresentazioni, Efficienza e Sicurezza

## Content Negotiation

### Principi fondamentali

La **content negotiation** consente a un client di esprimere preferenze rispetto alle rappresentazioni di una risorsa. Il client comunica queste preferenze attraverso specifici header della richiesta, permettendo al server di scegliere una rappresentazione adatta tra quelle disponibili.

### Pesi e fallback

Le preferenze possono essere assegnate con un livello di priorità tramite il parametro `q` (quality factor).

**Example**

HTTP

```http
GET /sections/wisdom.txt HTTP/1.1
Host: book-of-programming.local
Accept: text/plain
Accept-Language: en, it;q=0.9, fr;q=0.8
Accept-Encoding: gzip, deflate, br
Accept-Charset: utf-8, iso-8859-1;q=0.7, *;q=0.7
```

Una possibile risposta è:

HTTP

```http
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Language: it
Content-Encoding: br
Content-Charset: utf-8

Contenuto in italiano.
```

Il server non è obbligato a disporre di tutte le rappresentazioni richieste; se nessuna rappresentazione è accettabile per il client, il server può rispondere con uno status **406 Not Acceptable**. In pratica, tuttavia, i server spesso scelgono un fallback predefinito, ad esempio servendo la versione in una lingua di default se quella richiesta non è disponibile.

### Header di negoziazione comuni

I principali header di negoziazione includono `Accept` (tipo di contenuto), `Accept-Language` (lingua), `Accept-Encoding` (compressione) e `Accept-Charset` (set di caratteri). Ogni header comunica al server quale forma della risorsa il client preferisce.

### Content Encoding

L'header `Accept-Encoding` della richiesta indica gli algoritmi di compressione lossless che il client è in grado di decodificare. La compressione ha un impatto significativo sulla quantità di dati trasmessi, soprattutto per file di testo di grandi dimensioni. L'header **identity** indica l'assenza di compressione, invece, gli algoritmi più utilizzati sono: - **gzip** (basato su Lempel-Ziv coding – LZ77), - **br** (Brotli), - **deflate** (che impiega la struttura zlib con l'algoritmo deflate).

**Impatto della compressione** Il beneficio della compressione aumenta con la dimensione del file, va però ricordato che comprimere e decomprimere introduce a sua volta un overhead.

# Caching

La cache immagazzina una risposta affinché possa essere riutilizzata per richieste successive.

### Benefici

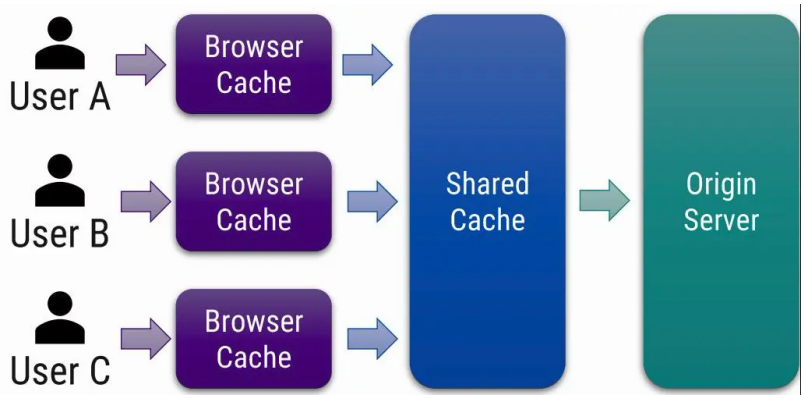
- **Latenza ridotta**: la risposta può essere disponibile più vicina al client.
- **Meno traffico di rete**: la stessa rappresentazione non viene ritrasmessa.
- **Meno lavoro per il server**: un numero inferiore di richieste raggiunge l'origine.
- **Scalabilità**: le cache condivise possono servire molti client.
- **Resilienza**: le cache possono servire risposte memorizzate quando l'origine è temporaneamente non disponibile.

### Problematiche critiche della cache

Il caching non è banale, perché non tutte le risposte vanno memorizzate I metodi **safe** (GET, HEAD) sono sempre cacheabili, mentre PUT e DELETE non lo sono; POST e PATCH sono teoricamente cacheabili se la risposta include header specifici, ma in pratica lo sono raramente.

### Livelli, cache private e condivise

Le risposte possono essere memorizzate in più punti lungo il percorso della richiesta, quindi una risposta può raggiungere il browser senza che il server di origine la elabori. Una cache **privata** serve un singolo utente, come la cache del browser; una cache **condivisa** può riusare la stessa risposta per più utenti, il che è appropriato per i contenuti pubblici ma non per i dati personali, che non devono mai finire per caso a un altro utente. Si noti che **se una risposta può essere memorizzata e chi può riusarla sono due decisioni distinte**.




### Il dilemma della consistenza

La caching introduce un **problema di consistenza**: le risposte memorizzate rappresentano lo stato della risorsa in un momento precedente. La cache deve bilanciare due obiettivi conflittuali: riutilizzare le risposte il più possibile per migliorare le prestazioni, ed evitare di servire risposte inaccettabilmente vecchie o inadatte. Questa tensione è al cuore della progettazione delle strategie di caching.

### Tre decisioni fondamentali

Per ogni risposta, la cache decide tre cose: se può **memorizzarla**, per quanto tempo può **riusarla senza validazione** e, quando il riuso diretto non è permesso, come sapere se la propria copia è ancora attuale mediante **validazione**. L'esito dipende dalla risposta specifica ed è controllato dai suoi **metadati**.

### L'header Cache-Control

L'header **Cache-Control** trasporta direttive che controllano il comportamento di caching. Le direttive sono separate da virgole; alcune ammettono un valore, altre no, e i tempi sono espressi in secondi.

| Direttiva | Significato |
|---|---|
| max-age=300 | La risposta è fresca per 300 secondi. |
| s-max-age=300 | Durata di freschezza per le cache condivise. |
| public | La risposta può essere memorizzata in cache pubbliche. |
| private | La risposta è destinata a una cache privata. |
| no-cache | La risposta può essere memorizzata, ma va validata prima di ogni riuso. |
| no-store | La risposta non va memorizzata in nessuna cache. |
| must-revalidate | Scaduta la freschezza, la risposta va validata prima del riuso. |
| immutable | La rappresentazione non cambierà durante la durata di freschezza. |

### Freschezza

```HTTP
HTTP/1.1 200 OK
Date: Thu, 13 Aug 2026 08:00:00 GMT
Cache-Control: public, max-age=300
```

Mentre una risposta è **fresca** può essere riusata direttamente senza validazione; non appena diventa **stale**, richiede in genere la validazione. La freschezza si calcola dai metadati della risposta — ad esempio `Date` più `max-age` — quindi una risposta può essere memorizzata in cache quando è già scaduta. In assenza di politiche esplicite le cache possono ricorrere a euristiche; per questo i server dovrebbero sempre fornire istruzioni di caching esplicite.

### Validatori: ETag e Last-Modified

Un server può fornire **validatori** nelle sue risposte per supportare una validazione efficiente. - L'**ETag** (Entity Tag) è un identificatore opaco scelto dal server che cambia quando la rappresentazione cambia. È il validatore più preciso e affidabile. - **Last-Modified** indica quando il server ritiene che la rappresentazione sia stata modificata per l'ultima volta; è meno preciso di ETag ma comunque utile.

```HTTP
ETag: "article-v7"
Last-Modified: Thu, 13 Aug 2026 08:15:00 GMT
```

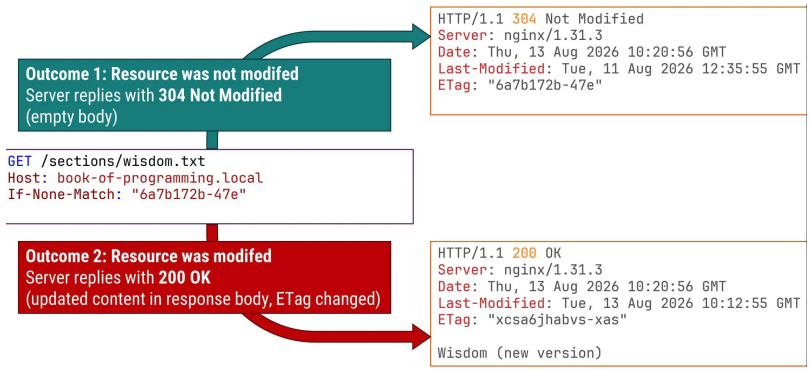
**Definition**

**Identificatore opaco**

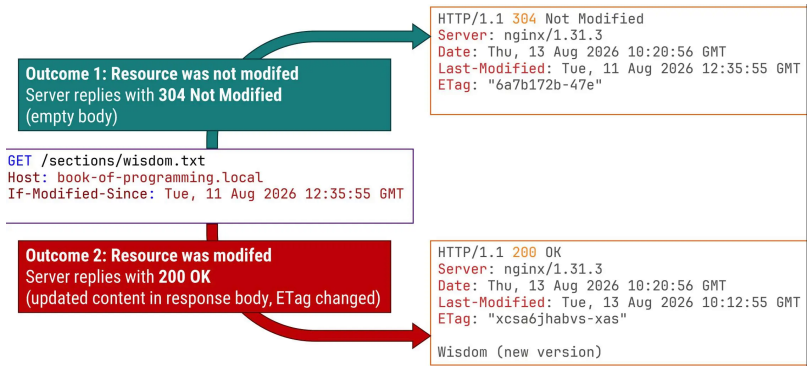
Il client non ha bisogno di conoscerne il significato interno né come è stata generata

### Validazione tramite conditional requests

La validazione avviene tramite **conditional requests**, ovvero richieste normali che includono header di validazione aggiuntivi. Quando una risposta è diventata *stale*, ma aveva un ETag o un Last-Modified, la cache può inviare una richiesta condizionale. Se la cache include l'header `If-None-Match: "article-v7"` e il server conferma che l'ETag è ancora valido, il server risponde con **304 Not Modified** (corpo vuoto), e la cache riusa il corpo memorizzato. Se la risposta è stata modificata, il server risponde con **200 OK** e il nuovo contenuto.



Analogamente, `If-Modified-Since` consente di validare basandosi sulla data di modifica.




### Caching e Content Negotiation

La **content negotiation** può creare varianti della stessa risposta. Un server può fornire dati compressi e non compressi per lo stesso URL. La cache potrebbe necessitare di archiviare risposte separate per ogni variante. Lo stesso accade per altre caratteristiche negoziate come `Accept-Language` e `Accept`. Quando il client invia una richiesta con `Accept-Language: it` non si può servire una risposta memorizzata con `Content-Language: en`.

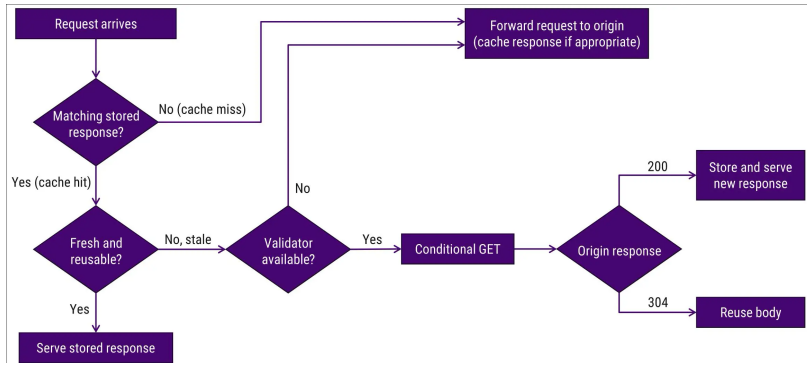
#### L'header Vary

Per risolvere questo problema, i server devono utilizzare l'header `Vary` per indicare quali header di richiesta influenzano la risposta.

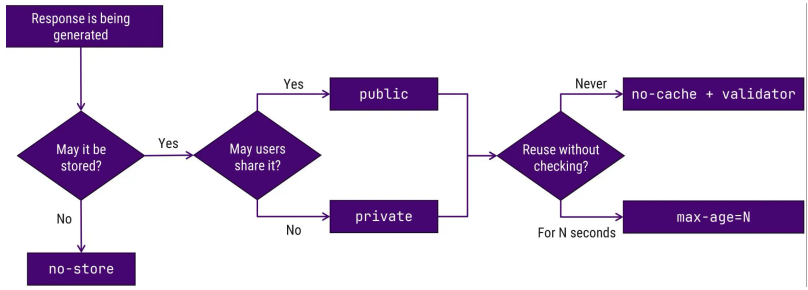
Ciò significa che una risposta precedente può essere riutilizzata per una richiesta successiva solo se la richiesta originale e quella attuale condividono lo stesso URL e metodo, e la richiesta attuale corrisponde all'header Vary della risposta vecchia.

#### Flusso decisionale della cache

- Cache view:




- Server view:




# HTTPS e Sicurezza della Comunicazione

Con HTTP un attaccante in grado di osservare o alterare la comunicazione può **leggere** richieste e risposte, rubando credenziali o identificatori di sessione, e può **modificare** le risposte, reindirizzando l'utente o impersonando il server legittimo. **HTTPS** affronta queste minacce a livello di comunicazione.

### HTTPS e TLS

HTTPS ha la **stessa semantica** di HTTP ma usa tipicamente la porta 443 invece della 80: è in sostanza HTTP su un canale **criptato**, la cui sicurezza è garantita da **TLS/SSL**. Concettualmente il funzinamento è questo:

1. Il client richiede `https://squids.unina.it`.
2. Il server presenta un **certificato** contenente gli hostname validi e una chiave pubblica.
3. Il client verifica catena di fiducia, validità temporale e hostname del certificato.
4. Il server dimostra di controllare la corrispondente chiave privata.
5. Viene stabilito un canale cifrato e protetto da integrità.

Con TLS 1.3, dopo il *ServerHello* l'handshake e tutto lo scambio HTTP successivo sono cifrati: un intercettatore vede ancora indirizzi IP, direzioni, tempi e dimensioni dei pacchetti, ma non può leggere metodo, URL, header né corpi delle risposte.

### Proprietà e limiti

HTTPS garantisce tre proprietà fondamentali: - **riservatezza** (chi ascolta il canale non può leggere le richieste), - **integrità** (nessuno può alterare richieste e risposte) , - **autenticazione del server** (il client verifica l'identità associata all'hostname).

Non bisogna però fidarsi troppo del "lucchetto verde": HTTPS garantisce **solo** la cifratura della comunicazione. Non assicura che il sito non sia malevolo, che le informazioni pubblicate siano corrette, che il server non sia compromesso, né che i dati siano al sicuro prima della trasmissione o dopo il loro arrivo al server.

# HTTP/2: Multiplexing e Efficienza

### Limitazioni di HTTP/1.1 con richieste parallele

Le moderne pagine web richiedono dozzine o centinaia di richieste per essere renderizzate in un browser. HTTP/1.1 può riusare una connessione TCP persistente, ma le risposte non possono essere liberamente interleaved: devono arrivare nell'ordine delle richieste. Ciò significa che una risposta lenta può ritardare tutte le risposte successive. I browser aperti tipicamente molteplici connessioni TCP verso lo stesso server per mitigare questo problema, il che comporta overhead di stabilimento e mantenimento di più connessioni, setup di trasporto e TLS ripetuti, competizione tra connessioni e un utilizzo inefficiente della capacità di rete.

### Multiplexing in HTTP/2

HTTP/2 mira a risolvere queste limitazioni. I messaggi sono rappresentati utilizzando **frame binari**. Ogni coppia richiesta-risposta è assegnata a uno **stream**. I frame provenienti da stream diversi possono essere interleaved, consentendo una serializzazione efficiente su un'unica connessione TCP. La semantica di più richieste HTTP è preservata: le richieste concettualmente rimangono `GET /index.html`, `GET /logo.png`, `GET /style.css`. L'unica differenza è che non sono trasmesse utilizzando la sintassi testuale di HTTP/1.1.

**Example**

Stream A: A1 A2 A3 Stream B: B1 B2 Stream C: C1 C2
Possible serialization over a single TCP connection A1 B1 B2 C1 A2 C2 A3
I frame A1, B1, B2, C1, A2, C2, A3 provenienti da tre stream (A, B, C) possono essere transmessi in questa sequenza mischiata.

### Head-of-Line Blocking e limitazione di TCP

HTTP/2 multiplessa gli stream a livello HTTP, ma TCP non può vedere questi stream indipendenti. TCP presenta un unico flusso di byte affidabile e ordinato; se un segmento TCP è perso, deve essere recuperato prima di consegnare i seguenti.

Questo collo di bottiglia è detto **Head-of-Line (HoL) Blocking**: un singolo elemento ritardato o bloccato arresta gli elementi successivi dal progredire, anche se questi ultimi sono indipendenti.

**Example**

Supponiamo tre streams: $A = (A1, A2) : B = (B1, B2) : C = (C1, C2)$
TCP Segment
Conceptual HTTP/2 bytes
Result
Segment 1 (S1)
A1 and part of B1
Received
Segment 2 (S2)
Rest of B1 and C1
Lost
Segment 3 (S3)
B2 and A2
Received but buffered (waiting for Segment 2)
HTTP/2 non può ricevere lo stream A sebbene A1 e A2 siano stati ricevuti, poiché S2 non contiene dati di A.

# HTTP/3 e QUIC

HTTP/3 è stato introdotto per mantenere lo stile di multiplexing di HTTP/2, evitando tuttavia il collo di bottiglia del TCP Head-of-Line blocking. HTTP/3 trasporta HTTP su **QUIC** anziché direttamente su TCP.

### Caratteristiche di QUIC

QUIC è costruito su **UDP**, fornisce **stream indipendenti multipli**, garantisce affidabilità e controllo della congestione, integra il handshake TLS 1.3 e è supportato da tutti i principali browser. È stato adottato dal 35-40% dei siti web principali.

# Parte III

# HTML

Capitolo 4

# Lezione 4 HTML: Hypertext Markup Language

## Web e HTML

- Il **web** è un sistema di documenti **ipertesto** interconnessi tramite **hyperlink**, recuperati dai client (browser) tramite HTTP.
- Un **server HTTP** è un software in ascolto su una porta che può servire più host (da qui l'header **Host**) e distribuisce i file a partire dalla **document root** del proprio filesystem.
- Una **Web App** consiste di una o più pagine web, descritte in HTML e visualizzate dal browser.

Definition

**Linguaggio di markup**

Un **linguaggio di markup** arricchisce un documento con annotazioni (**tag**, tra parentesi angolari) che ne controllano struttura, formattazione o relazioni tra le parti; i tag di apertura possono contenere **attributi** chiave-valore con valore opzionale.

```HTML
<tagName attr1="value" attr2> ... <\tagName>
```

- HTML esiste dal 1993 in diverse versioni; il riferimento attuale è l'**HTML Living Standard**.

### Struttura del documento

- Il documento inizia con la dichiarazione `<!DOCTYPE HTML>`: non è un tag, indica al client il tipo di documento atteso.
- Il tag `<html>` rappresenta l'intero documento e contiene un `<head>` e un `<body>`; l'attributo **lang** è raccomandato per l'accessibilità.
- Lo `<head>` contiene i **metadati** (dati sul documento, spesso non mostrati all'utente ma utili a browser e motori di ricerca) e deve contenere un `<title>`.
- Lo `<body>` contiene il contenuto effettivo del documento.

Definition

**Attributi globali**

Alcuni attributi sono **globali**, cioè ammessi su qualunque elemento; altri hanno senso solo per alcuni elementi. - **id**: identificatore univoco dell'elemento nel documento. - **lang**: lingua del contenuto dell'elemento. - **style, class**: usati per lo stile.

### Commenti

I **commenti** sono ignorati dai browser, delimitati da `<!--` e `-->`; servono per note o per nascondere temporaneamente contenuto.

# Elementi fondamentali

### Titoli e paragrafi

- `<h1>`–`<h6>` rappresentano titoli e sottotitoli di livello crescente.
- `<p>` rappresenta un paragrafo, che tipicamente inizia su una nuova riga.

### Semantica a livello di testo

- `<em>` enfatizza il contenuto; `<strong>` indica forte importanza; `<br>` è un elemento *void* che inserisce un'interruzione di riga.
- `<abbr title="...">` definisce acronimi e abbreviazioni.
- `<del>` marca contenuto eliminato dal documento; `<ins>` contenuto inserito.

### Ancore e URL

- I **link** si definiscono con l'ancora `<a>`; l'attributo **href** punta all'URL di destinazione.
- Un URL è composto da **schema** (protocollo), **nome di dominio**, **porta** e **percorso**; può includere una **query string** e un **anchor**.

### URL assoluti e relativi

- Un **URL assoluto** include schema e hostname e contiene tutte le informazioni per raggiungere la risorsa.
- Un **URL relativo** specifica solo un percorso; schema e hostname sono dedotti dal contesto corrente.
- Un URL relativo che inizia con `/` sostituisce l'intero percorso; altrimenti sostituisce solo l'ultimo segmento del percorso.
- I segmenti **dot** indicano directory: `.` è la directory corrente, `..` la directory padre.

| href | Percorso risultante (da `/a/b/c/hello.html`) |
|---|---|
| `./index.html` | `/a/b/c/index.html` |
| `../foo.html` | `/a/b/foo.html` |
| `../../pic.jpg` | `/a/pic.jpg` |

- Le URL relative sono da preferire per link interni alla stessa web app (cambiare l'hostname non richiede modifiche alle pagine); per risorse esterne servono URL assoluti.
- L'attributo **target** specifica dove aprire la risorsa: `_self` (default) nella stessa scheda, `_blank` in una nuova scheda/finestra.

### Anchor e index.html

- Un **anchor** (`#id`) è un "segnalibro" dentro la risorsa: indica al browser di mostrare il contenuto in corrispondenza dell'elemento con quell'`id`.
- Quando un URL punta a una directory, il server risponde tipicamente con il file `index.html` in essa (comportamento configurabile: anche `home.html`, `default.htm`, o generazione automatica di un indice).

### Tabelle

- `<table>` contiene righe `<tr>`; ogni riga contiene celle `<td>` o intestazioni `<th>`.
- `<thead>`, `<tbody>` e `<tfoot>` raggruppano rispettivamente intestazioni di colonna, righe di dati e righe di riepilogo; `<caption>` descrive la tabella nel suo insieme.

### Liste

- **Liste ordinate** `<ol>`: per enumerazioni.
- **Liste non ordinate** `<ul>`: per elenchi puntati; entrambe contengono elementi `<li>`.
- **Liste di descrizione** `<dl>`: coppie di termine `<dt>` e descrizione `<dd>`, spesso usate per glossari.

### Riferimenti di carattere

- Alcuni caratteri (es. `<`, `>`) sono **riservati** in HTML: per visualizzarli si usano i **riferimenti di carattere** (entità) nella forma `&nome;` o `&#numero;`.

| Risultato | Riferimento |
|---|---|
| Spazio non separabile | ` ` |
| < | `<` |
| > | `>` |
| & | `&` |
| " | `"` |
| ' | `'` |
| © | `©` |

### Immagini

- `<img>` è un elemento *void* che incorpora un'immagine: **src** specifica l'URL, **alt** il testo alternativo, **width**/**height** le dimensioni in pixel.
- `alt` è un sostituto testuale che deve esprimere significato o scopo dell'immagine: sostituendo mentalmente l'immagine col suo testo, la pagina deve comunicare la stessa informazione. > Non usare descrizioni generiche: lo scopo è far trasmettere dagli screen reader lo stesso significato o funzione a chi non vede le immagini.
- Non usare descrizioni generiche: lo scopo è far trasmettere dagli screen reader lo stesso significato o funzione a chi non vede le immagini.

Info

**Testo alt in base allo scopo**

- Immagine **informativa** → testo alternativo significativo. - Immagine **decorativa** → `alt=""` (vuoto). - Immagine che è l'unico contenuto di un link → `alt` che descrive la destinazione o lo scopo del link.

Info

**Dietro le quinte**

I documenti HTML erano finora **autonomi** (self-contained). Con `<img>` il contenuto esterno è indicato solo tramite URL: l'immagine non è inclusa nel documento, quindi il browser, che **parsa dall'alto verso il basso**, deve **recuperare (fetch)** risorse aggiuntive per visualizzarlo.

# Form

- L'elemento **<form>** raccoglie input dell'utente, tipicamente inviati a un server; contiene controlli come <input>, <label>, <textarea>, <select>.

Definition

**Controlli successful e form data set**

All'invio, il **form data set** è costruito raccogliendo tutti i **controlli successful** del form: coppie nome/valore separate da &, dove il nome è l'attributo name di un input e il valore è quello assunto al momento dell'invio.

**Condizioni per un controllo successful:** - Deve essere definito dentro un <form> e avere un attributo **name**. - Un controllo **disabled** non è successful (ignorato nell'invio). - Con più pulsanti di invio, solo quello attivato è successful. - Tutte le checkbox selezionate possono essere successful. - Per i radio button con lo stesso name, solo quello selezionato è successful. - L'algoritmo completo è nell'HTML Living Standard.

### Controlli di input

- Il tipo di <input> è definito dall'attributo **type**: text, password, number, radio (una scelta tra molte), checkbox (zero o più scelte), submit (pulsante), oltre a tipi dedicati a date e orari (date, week, month, time, datetime-local) e altri (color picker, file picker, datalist…).
- <select> definisce menu a tendina con <option>; l'attributo **multiple** permette la selezione multipla.
- L'attributo **disabled** fa sì che un controllo sia ignorato durante l'invio.

### Label e raggruppamento

- **<label>** etichetta un input: l'attributo **for** deve valere l'id dell'input corrispondente; usarle è una buona pratica di usabilità e accessibilità.
- **<fieldset>** raggruppa logicamente i controlli, con **<legend>** come titolo/caption del gruppo.

### Invio (submission)

Alla sottomissione si genera una nuova richiesta HTTP verso il **form-handler** indicato dall'attributo: - **action** (default: URL della pagina corrente); - **method** indica il metodo HTTP (default: GET).

HTML

```
<form action="/handler.html" method="GET">
Message: <input type="text" name="msg"><br>
Number: <input type="number" name="num"><br>
<input type="submit">
</form>
```

I dati raccolti formano il **form data set**: coppie nome/valore name1=value1&...&nameN=valueN , dove il nome è l'attributo **name** di ciascun controllo e il valore il suo contenuto all'atto dell'invio.

**Method**

- Con **GET** i dati sono accodati alla URL del handler come **query string** (`/handler.html?msg=Hello!&num=42`): i nomi sono detti **query parameters**, coppie chiave/valore separate da `&`, che il server può usare prima di restituire la risorsa.
- Con **POST** i dati sono inviati nel corpo della richiesta.

**URL Encoding**

La **URL encoding** sostituisce i caratteri speciali con terne `%XX`, dove XX sono due cifre esadecimali che rappresentano il carattere ASCII (gli spazi diventano `%20` o `+`).




### Validazione nativa

- I browser moderni hanno **validazione integrata**: verificano vincoli sull'input e bloccano l'invio se non soddisfatti.
- Vincoli esprimibili con attributi: **type** (tipo atteso), **required** (valore non vuoto), **minlength/maxlength** (lunghezza del testo), **min/max/step** (numeri e date), **pattern** (espressione regolare da rispettare).

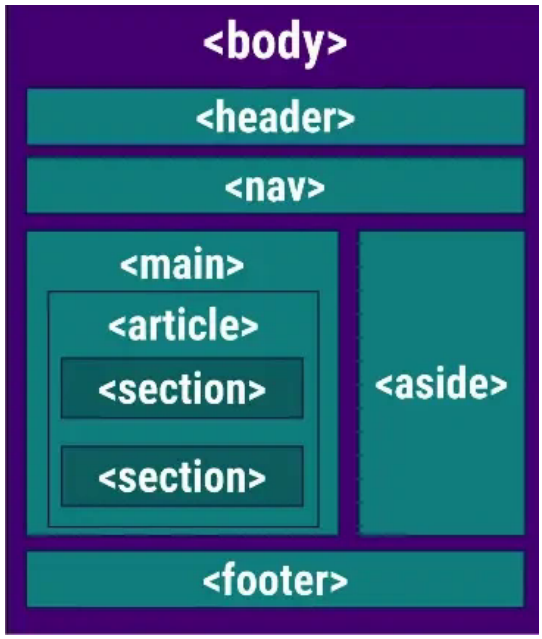
Warning

**La validazione HTML non è sicurezza**

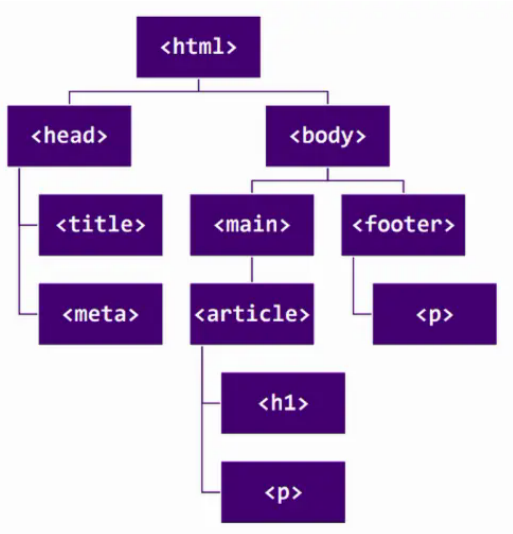
La validazione gira nel browser dell'utente, che potrebbe usare un browser senza validazione, modificare l'HTML con le DevTools o inviare richieste HTTP direttamente: un client malintenzionato può inviare qualunque valore. Restano comunque utili per intercettare gli errori presto e migliorare l'esperienza utente.

# Organizzazione del contenuto

Il contenuto di una pagina può essere raggruppato con **divisioni** `<div>` (nessuna semantica oltre al raggruppamento) o con **tag semantici**, che descrivono il significato del contenuto a browser, sviluppatori e software. - `<nav>` contiene link di navigazione; - `<main>` il contenuto principale; - `<article>` contenuto indipendente e autonomo; - `<aside>` contenuto tangenzialmente correlato; - `<header>`, `<footer>`, `<section>` autoesplicativi.




Un documento HTML è un **albero**: `html` è la radice, con figli `head` e `body`; il nesting di elementi definisce la gerarchia.




## Errori HTML e recupero dei browser

- I browser cercano di visualizzare la pagina anche in presenza di errori (**tag soup**): chiudono automaticamente tag non chiusi, aggiungono markup mancante, correggono entità e problemi strutturali.

- Questo può mascherare errori durante l'apprendimento: una pagina visualizzata correttamente non implica HTML sintatticamente o semanticamente corretto; conviene usare linter (es. HTMLHint) o validatori online (es. validator.w3.org/nu).

## DevTools

- La scheda "Inspector"/"Analisi pagina" dei DevTools del browser permette di ispezionare l'HTML del documento e modificarlo **localmente**, cioè solo nella copia caricata nel browser, non sul server.

# Parte IV

# CSS

Capitolo 5

# Lezione 05 CSS: Cascading Style Sheets

## Ruolo del CSS nel web

- **HTML** si occupa della struttura e della semantica dei documenti: non dice nulla sull'aspetto (appearance) dei contenuti.
- Un elemento come `<em>` specifica solo che il contenuto va enfatizzato, non come: l'enfasi può essere resa con corsivo, colori o sfondi diversi.

Definition

**CSS**

Linguaggio **dichiarativo e basato su regole** che specifica come i documenti vengono presentati agli utenti.

- Un **stylesheet** è un insieme di **regole**; ogni regola è composta da un **selettore** (quali elementi HTML sono coinvolti) e da un blocco di **dichiarazioni** (coppie proprietà-valore che specificano lo stile da applicare).

```css
selector {
    property: value;
    property: value;
}
```

### Stili di default del browser (user agent styles)

- Il browser applica a ogni pagina dei suoi stili di base, detti **user agent styles**: titoli più grandi e in grassetto, `<p>` a capo riga, `<em>` in corsivo, `<strong>` in grassetto, `<a>` sottolineati e blu, `<ul>` con elenchi puntati, ecc.
- Questi default sono all'incirca uguali tra i vari browser, ma esistono alcune differenze.

## Includere stili in una pagina

- **`<link rel="stylesheet" href="...">`** nel `<head>`: collega un foglio di stile esterno; `rel` specifica la relazione con il documento collegato, `href` l'URL del foglio. Come per `<img>`, il browser fa una richiesta HTTP aggiuntiva per recuperare il foglio prima del rendering.
- **`<style>`** nel `<head>`: definisce regole CSS direttamente dentro il documento HTML; è in genere preferibile usare fogli esterni e `<link>`.
- **Stili inline**: attributo `style` su un singolo elemento; il valore è una sequenza di dichiarazioni separate da `;` e vale solo per l'elemento che lo porta.

```HTML
<em style="color: fuchsia; font-weight: bold;">inline style</em>
```

# Selettori

**Info**

**Usi dei selettori oltre allo stile**

I selettori CSS non servono solo per lo stile: con **JavaScript** selezionano gli elementi con cui interagire, nel **testing automatizzato** del web individuano gli elementi su cui agiscono i test, e nello **scraping/crawling** selezionano gli elementi che contengono le informazioni da estrarre.

### Selettori semplici

- **Selettore universale** (wildcard): `*`, corrisponde a qualsiasi elemento.
- **Selettore di tipo**: il nome del tag; corrisponde a tutti gli elementi di quel tipo (es. `a`).
- **Selettore di id**: forma `#ElementId`, corrisponde all'elemento con quell'attributo `id`.
- **Selettore di classe**: forma `.classname`, corrisponde agli elementi con quell'attributo `class` (un elemneto può avere più classi, limportante che siano separati da uno spazio `class=".classname1 classname2 ..."` ).
- **Selettore di attributo**: forma `[attribute]` o `[attribute='value']`, corrisponde agli elementi con quell'attributo (o con quel valore).

```CSS
[for] { background: yellow; }        /* tutti gli elementi con attributo for */
[type='number'] { background: cyan; } /* tutti gli elementi con type=number */
```

### Operatori di corrispondenza parziale sui valori degli attributi

- `[attribute*='v']`: il valore **contiene** `v`.
- `[attribute^='v']`: il valore **inizia con** `v`.
- `[attribute$='v']`: il valore **termina con** `v`.

```CSS
[href^='https'] { color: red; }  /* inizia con https */
[href$='.it/']  { color: green; } /* termina con .it/ */
```

### Selettori composti (compound)

- Si **concatenano** selettori senza spazio per ottenere un controllo fine: la selezione è l'**intersezione** dei selettori coinvolti (es. `a[href*='programming'].my-class` corrisponde agli `a` il cui `href` contiene "programming" e che hanno la classe `my-class`).

### Combinatori

- I **combinatori** selezionano elementi in base alla loro posizione nel documento (un documento HTML è un albero); sintassi: `selector1 combinator selector2`.

| Combinatore | Nome | Semantica |
|---|---|---|
| (spazio) | Selettore discendente | `A B` corrisponde a tutti gli elementi che soddisfano `B` contenuti (discendenti) in un elemento `A` |
| `>` | Selettore figlio | `A > B` corrisponde a tutti gli elementi `B` figli **diretti** di `A` |
| `+` | Selettore di fratelli adiacenti | `A + B` corrisponde agli elementi `B` fratelli **immediatamente successivi** a `A` |
| `~` | Selettore generale di fratelli | `A ~ B` corrisponde agli elementi `B` fratelli **successivi** (anche non adiacenti) a `A` |

```css
section em { color: teal; }        /* discendente */
main > em  { color: teal; }         /* figlio diretto */
.master + li { color: red; }        /* fratello adiacente */
.master ~ li.disciple { color: red; } /* fratelli successivi */
```

### Pseudo-classi

**Definition**

**Pseudo-classe**

I elementi HTML possono trovarsi in **stati** diversi (per interazione dell'utente o per relazione con altri elementi); le pseudo-classi, con sintassi che inizia con `:`, permettono di stilizzare gli elementi in base al loro stato.

- **Stati interattivi** (derivanti dall'interazione utente): `:hover` (puntatore del mouse sopra l'elemento), `:active` (elemento attivamente in interazione, es. pulsante premuto), `:focus` (elemento attualmente selezionato/fuocato, es. link o campo input).
- **Stati storici** (memorizzano i link visitati): `:link` (link non ancora visitati), `:visited` (link già visitati). Nota: gli stati storici non sono supportati nel Live Preview di VS Code; vanno verificati in un browser reale.
- **Stati dei form** (specifici dell'interazione con i form): `:disabled`, `:invalid`, `:checked`.
- **Stati di posizione** (relazioni con gli altri elementi tra fratelli, siblings):
  - `:first-child` e `:last-child`: primo/ultimo figlio tra un insieme di fratelli.
  - `:only-child`: elementi senza fratelli.
  - `:first-of-type` e `:last-of-type`: primo/ultimo fratello **considerando solo gli elementi dello stesso tipo**.
  - `:nth-child(n)` e `:nth-of-type(n)`: elemento in ennesima posizione tra i fratelli; accettano anche parole chiave come `even` e `odd`.

Warning

**Indicizzazione**

In CSS l'indicizzazione parte da **1**, non da 0.

```css
em:last-of-type { color: red; }
li:nth-child(even) { color: red; }
li:nth-child(odd)  { color: blue; }
```

### Pseudo-elementi

Definition

**Pseudo-elemento**

Pseudo-elemento (sintassi `selector::pseudo-element`) che permette di mirare **parti specifiche del contenuto** di un elemento HTML, senza aggiungere markup HTML aggiuntivo.

- `::first-letter`: la prima lettera del contenuto di un elemento a livello di blocco (block-level).
- `::first-line`: la prima riga del contenuto di un elemento block-level.
- `::selection`: il contenuto attualmente selezionato dall'utente.
- `::before`: crea un elemento che è il **primo figlio** dell'elemento selezionato.
- `::after`: crea un elemento che è l'**ultimo figlio** dell'elemento selezionato.

```css
p::first-letter { font-weight: bold; }
.narrator::before {
    content: "Narrator >> ";
    font-weight: bold;
}
```

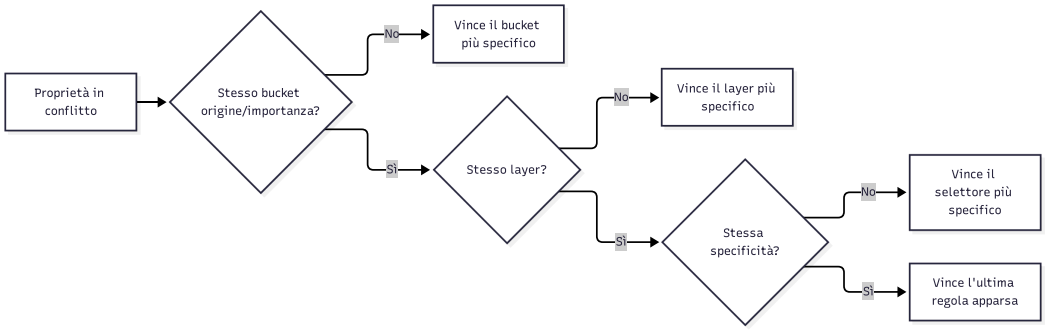
## La cascata (the cascade)

Definition

**La cascata**

Algoritmo usato per risolvere i **conflitti** quando due o più regole si applicano allo stesso elemento assegnando valori diversi alla stessa proprietà. Input: un insieme di proprietà conflittuali per un elemento; output: la singola proprietà (con valore) da applicare effettivamente.

- La cascata considera 4 aspetti chiave, in ordine: **origine e importanza**, **layers**, **specificità**, **posizione/ordine di apparizione** della regola.




### Origine e importanza

- Il CSS che scriviamo (**authored CSS**) non è l'unico applicato a una pagina: esistono gli **user agent styles** e altri stili (**local user styles**) aggiunti da estensioni del browser o dal sistema operativo, ad esempio per accessibilità (schemi a alto contrasto, font più grandi per ipovedenti).
- L'annotazione `!important`, aggiunta alla fine di una dichiarazione, conferisce più importanza a quella dichiarazione; l'importanza pesa in modo significativo nella cascata.

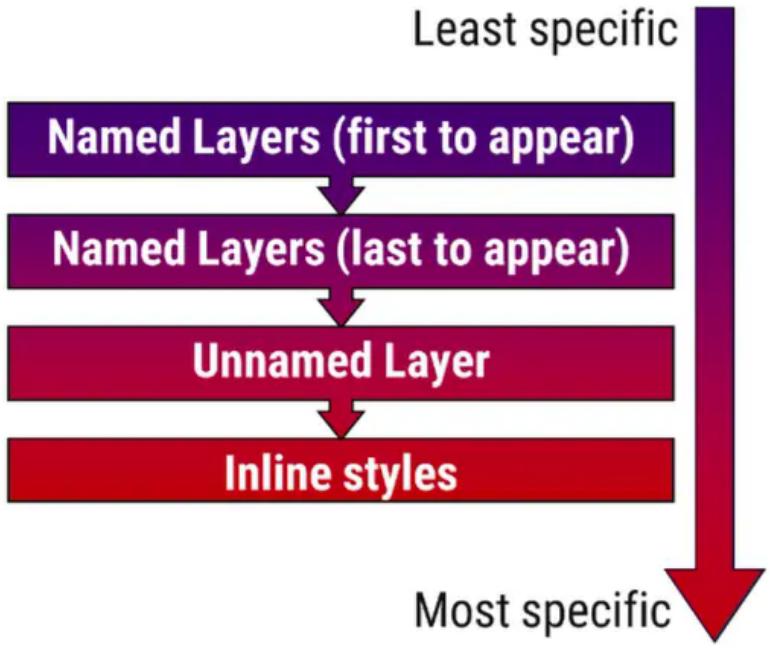
```css
h1 { color: red !important; }
```

- Gli origin bucket vanno dal **meno specifico al più specifico**; l'importanza (`!important`) crea ulteriori sotto-bucket all'interno di ciascuna origine.




### Layers

- Dentro ogni bucket origine/importanza possono esserci più **cascade layers**:
  - Negli stili authored si possono definire **layer personalizzati** con la regola `@layer`; i layer dichiarati **più tardi hanno priorità più alta**.
  - Tutto il CSS in `<style>` o importato con `<link>` appartiene a un **layer senza nome**.
  - Gli **stili inline** appartengono a un layer separato e hanno la **priorità più alta**.




### Specificità

**Definition**

**Specificità**

Una terna numerica **(A, B, C)** calcolata da un selettore: si ignora il selettore universale; **A** = numero di selettori di id; **B** = numero di selettori di classe, attributo e pseudo-classi; **C** = numero di selettori di tipo e pseudo-elementi.

- Il calcolo vale quando due regole in conflitto appartengono allo stesso bucket origine/importanza e allo stesso layer: **vince il selettore più specifico**.

| Selettore | Specificità (A, B, C) |
|---|---|
| `*` | (0, 0, 0) |
| `a:hover` | (0, 1, 1) |
| `article.item section p::first-letter` | (0, 1, 4) |
| `em.master[target]` | (0, 2, 1) |
| `#id` | (1, 0, 0) |
| `#navbar ul li a.nav-link[href*='/']` | (1, 2, 3) |

### Confronto tra specificità

- Le tre componenti si confrontano **in ordine**: vince la specificità con A maggiore; a parità di A, quella con B maggiore; a parità anche di B, quella con C maggiore; se tutti i valori sono pari, le specificità sono **uguali** (pareggio).

**Example**

| Selettore #1 | Spec. #1 | Selettore #2 | Spec. #2 | Vincitore |
|---|---|---|---|---|
| `a[target]` | (0, 1, 1) | `.list a` | (0, 1, 1) | Pareggio |
| `#msg` | (1, 0, 0) | `input[type].inp` | (0, 2, 1) | #1 |
| `#nav > #brd a.lk` | (2, 1, 1) | `em.foo.bar.light` | (0, 3, 1) | #1 |
| `[id='nav'] a` | (0, 1, 1) | `#nav a` | (1, 0, 1) | #2 |

**Posizione e ordine di apparizione**

- Se due regole appartengono allo stesso bucket origine/importanza, allo stesso layer e hanno la stessa specificità, **l'ultima regola apparsa ha la priorità più alta**.
- La regola vale sia dentro lo stesso stylesheet, sia in base all'ordine in cui i fogli di stile compaiono.

```css
h1 { color: red; }
h1 { color: blue; } /* vince blue */
```

# Ereditarietà (inheritance)

- Alcune proprietà CSS (**inheritable**) — ad esempio `color`, `font-size`, `font-family`, `font-weight`, `font-style` — vengono **ereditate dagli antenati** se all'elemento non è impostato un valore specifico.
- Le proprietà ereditate **non prendono parte alla cascata**: vengono considerate solo quando la cascata non ha prodotto alcun valore per quell'elemento; in un certo senso hanno la specificità più bassa di tutti i metodi di stilizzazione.

```css
p {
    font-family: sans-serif;
    color: red;
}
/* l'elemento <em> dentro <p> eredita color e font-family */
```

**Info**

**Ispezione nel browser**

Negli strumenti di sviluppo (Dev Tools) gli user agent styles sono **nascosti per default**: per vederli occorre premere F1 e cambiare l'impostazione.

# Lezione 06 CSS: Layouts, Responsive Design

## Unità di misura (sizing units)

### Panoramica

- Alcune proprietà CSS cambiano la **dimensione** degli elementi: `width`, `height`, `font-size`, `margin`, `padding`, `border`, …
- Per dimensionare gli elementi si usano lunghezze **assolute** o lunghezze **relative** (dipendono da altre dimensioni: font o viewport).

### Unità assolute

- Le lunghezze assolute si definiscono con un numero e una delle **unità di lunghezza** supportate.

```css
div {
    width: 2.54cm;  /* centimetri */
    height: 1in;    /* pollici */
    border: 2mm solid black; /* millimetri */
    font-size: 24px; /* pixel */
}
```

### Percentuali

- Le **percentuali** definiscono dimensioni relative all'elemento **padre**.
- Si usano con molte proprietà: `width`, `height`, `font-size`, `margin`, `padding`, …

```css
.b { width: 50%; height: 50%; } /* 50% del padre */
```

### Unità relative al font

- **em**: `1em` rappresenta la dimensione di font corrente (`1.5em` è il 50% in più); il nome deriva storicamente dalla larghezza della "M" maiuscola in tipografia.
- **rem**: `1rem` rappresenta il `font-size` calcolato all'elemento **radice** (default 16px).

```css
p { font-size: 1.5rem; }
span { font-size: 1.5em; }
```

### Unità relative al viewport

**Definition**

**Viewport**

Il **viewport** è la finestra del browser.

- **vw**: `1vw` rappresenta l'1% della **larghezza** del viewport corrente.
- **vh**: `1vh` rappresenta l'1% dell'**altezza** del viewport corrente.

```css
div { height: 50vh; }
.b { width: 50vw; font-size: 10vw; }
```

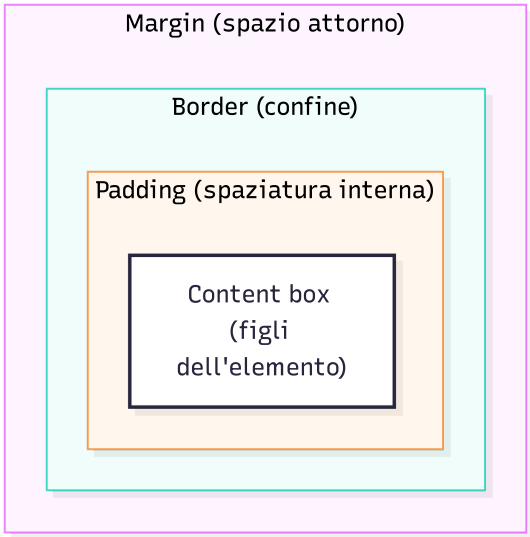
## Il box model

**Definition**

**Box model**

Ogni elemento HTML è un **box** composto da aree distinte.

- **Content box**: l'area dove vivono i figli dell'elemento.
- **Padding**: spaziatura interna, separa il content box dal bordo.
- **Border**: il confine dell'elemento.
- **Margin**: crea spazio attorno agli elementi.



- La dimensione di ogni area si definisce con dichiarazioni CSS.
- Comportamento e aspetto dei box sono determinati da: **layout mode**, **contenuto** e **proprietà del box model**.
- Se il contenuto non entra nel box, la parte in eccesso si chiama **overflow**.

# Layout

### Flow layout (normal flow)

Per default i box sono visualizzati con il **flow layout** (a.k.a. **normal flow**). Esistono due tipi di elementi: - **inline** si dispongono nella direzione inline, come parole in una frase. - **block** si dispongono uno dopo l'altro, ognuno su una nuova riga dedicata.

### La proprietà display

- `display: inline` posiziona il box accanto al precedente nella direzione inline; **ignora** `width` e `height`.
- `display: block` posiziona il box su una nuova riga dedicata; se non specificato, i block si espandono su tutta la dimensione inline.
- `display: inline-block` è come `inline`, ma **permette** di impostare `width` e `height`.
- `display: none` rimuove completamente l'elemento dalla visualizzazione.

```css
em { display: inline-block; width: 50px; height: 50px; }
```

### Floats

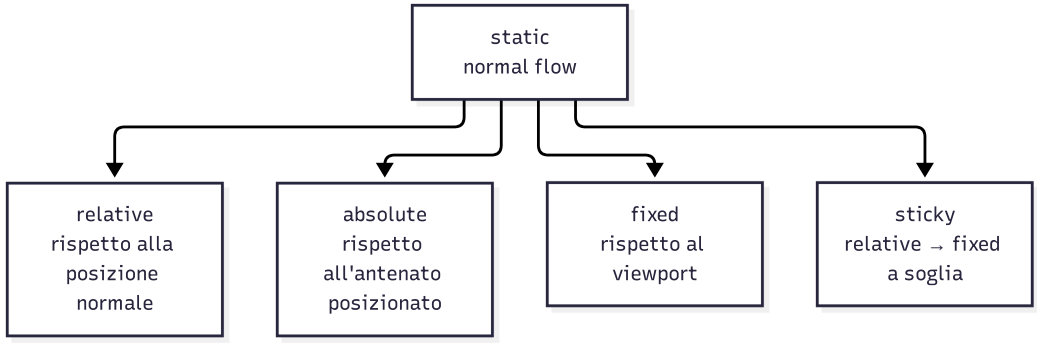
- La proprietà `float` fa "galleggiare" un elemento nella direzione indicata, con i fratelli successivi che vi **scorrono attorno** (wrap).
- La proprietà `clear` impedisce a un fratello successivo di fare wrap attorno a un elemento flottante.

```css
img { float: left; margin-right: 1rem; }
p+p { clear: both; }
```

### Positioning

- La proprietà `position` cambia il comportamento dell'elemento nel normal flow.
- Il valore di default è **static**; gli altri valori sono `relative`, `absolute`, `fixed`, `sticky`.
- **relative**: l'elemento è posizionato rispetto alla sua posizione normale.
- **absolute**: l'elemento è posizionato rispetto all'**antenato posizionato più vicino** (nearest relative-positioned ancestor).
- **fixed**: l'elemento è posizionato rispetto al **viewport**.

- **sticky**: ibrido relative/fixed — relativo finché non supera una soglia (`top`, …), poi fixed finché non raggiunge il confine del proprio padre.




CSS

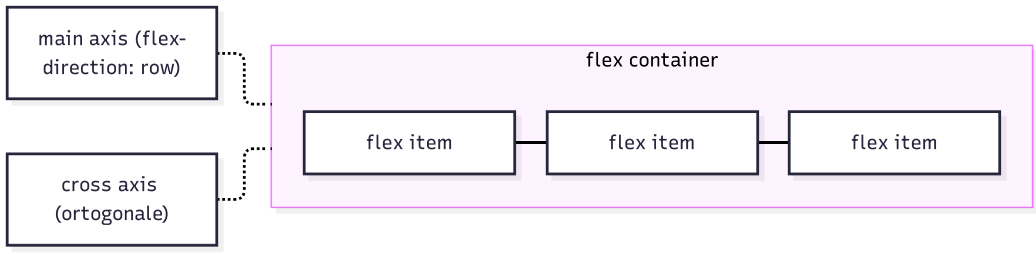
```
nav { position: sticky; top: 0px; }
a  { position: fixed; bottom: 2rem; right: 2rem; }
```

## Layout moderni: Flexbox e Grid

- Oltre al normal flow, il CSS moderno ha due meccanismi di layout: **Flexbox** e **Grid**.
- **Flexbox** è pensato per layout **unidimensionali** (orizzontale o verticale).
- **Grid** è pensato per layout **bidimensionali**.

### Flexbox: flex container

- I flex container si dichiarano con `display: flex`; sono elementi block-level con figli **flex item**.
- Ogni container ha un **main axis** e un **cross axis** (ortogonale).
- Il main axis è impostato da `flex-direction` (default: `row`).




### Flexbox: flex items

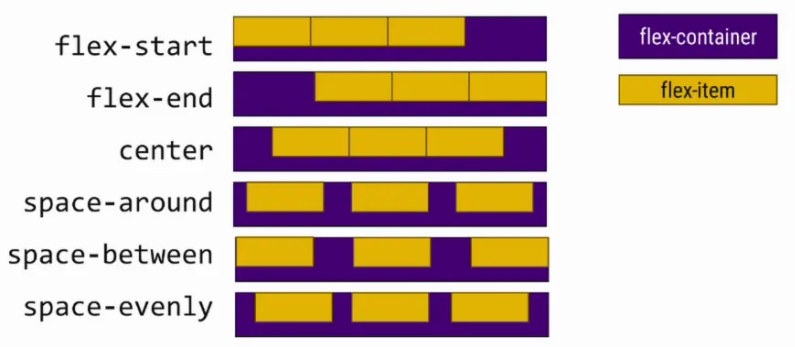
I figli del container diventano **flex item** e si collocano lungo il main axis. - **flex-grow**: fa espandere l'item in tutto lo spazio disponibile lungo il main axis. - **flex-shrink**: controlla se l'item può ridurre la propria dimensione di base per stare nel container.

Gli overflow sono possibili e si controllano con `flex-wrap` sul container.

**Flexbox: gestione dello spazio libero**

- **justify-content**: gestisce lo **spazio libero lungo il main axis**.
- **align-content**: gestisce lo **spazio libero lungo il cross axis**.
- Valori possibili includono: `flex-start`, `flex-end`, `center`, `space-between`, `space-around`.

```css
.container {
    display: flex;
    justify-content: space-around;
    align-items: center;
}
```




**Grid**

Grid è un layout **bidimensionale** basato su **righe e colonne**, si dichiarano con `display : grid`; i figli diretti sono **grid item**. Il container definisce **numero e dimensione** di righe e colonne (`grid-template-columns`, `grid-template-rows`, `gap`). Per default gli item sono collocati lungo le righe; `grid-auto-flow: column` li colloca lungo le colonne.

```css
.container {
    display: grid;
    grid-template-columns: 1fr 100px;
    grid-template-rows: 20px 50px 100px;
    gap: 10px;
}
```

**L'unità fr**

**fr** è un'unità relativa specializzata che funziona solo nei grid layout. Rappresenta una lunghezza flessibile corrispondente a una quota dello spazio disponibile, in modo simile alla proprietà `flex`. - Esempio: `grid-template-columns: 1fr 1fr 1fr` definisce tre colonne con la stessa quota di spazio.

### Template areas

- Si possono assegnare nomi ad **aree** (insiemi di celle) con `grid-template-areas` e collocare item in un'area con `grid-area`.

```css
body {
    display: grid;
    grid-template-columns: 70vw 1fr;
    grid-template-areas:
      "header header"
      "navbar navbar"
      "main sidebar"
      "footer footer";
}
header { grid-area: header; }
aside  { grid-area: sidebar; }
```

## Media queries

Definition

**Media query**

Una media query applica stili CSS solo quando il **dispositivo** che visualizza il contenuto ha **caratteristiche specifiche**.

- Le media query si aprono con la keyword `@media`.

```css
@media print {
    body { color: black; background: transparent; }
}
```

### Tipi di output

- **print**: materiale paginato e documenti in anteprima di stampa.
- **screen**: dispositivi che visualizzano il documento su schermo.
- **all**: tutti i dispositivi di output.

### Media features

- Le **media features** verificano caratteristiche dello user-agent, del dispositivo di output o dell'ambiente.
- Sintassi completa: `@media media_type and (media_feature: value) and ....`
- Se `media_type` è omesso si usa `all`; le feature vanno racchiuse tra parentesi e sono opzionali.

```css
@media screen and (orientation: landscape) { /* ... */ }
@media (min-width: 768px) { /* ... */ }
```

# Responsive design

### Fixed-width layouts

- Progettare per una singola larghezza specifica (640, 800, 1024px) si chiama **fixed-width design**.
- Funziona bene solo alla larghezza considerata: schermi più piccoli richiedono scrolling orizzontale, schermi più grandi mostrano molto spazio sprecato.

### Liquid (fluid) layouts

- I **liquid layouts** usano le **percentuali** come larghezza delle colonne, senza flexbox, grid o media queries.
- Reggono su un intervallo più ampio di larghezze, ma degradano agli estremi: **stretched** su schermi molto larghi, **squashed** su schermi molto stretti.

### Siti separati (mobile)

- Con l'arrivo dei dispositivi mobili (schermi fino a 240px) un'opzione era un **sito separato** per mobile.
- Tipicamente basato su **user agent sniffing** per reindirizzare gli utenti mobili a una versione su sottodominio.

Warning

**Contro dei siti separati**

- Lo UA sniffing è inaffidabile (può mancare i mobile o reindirizzare i desktop). - Ogni sito separato va mantenuto. - La distinzione mobile/non-mobile oggi è sfumata.

### Adaptive layouts

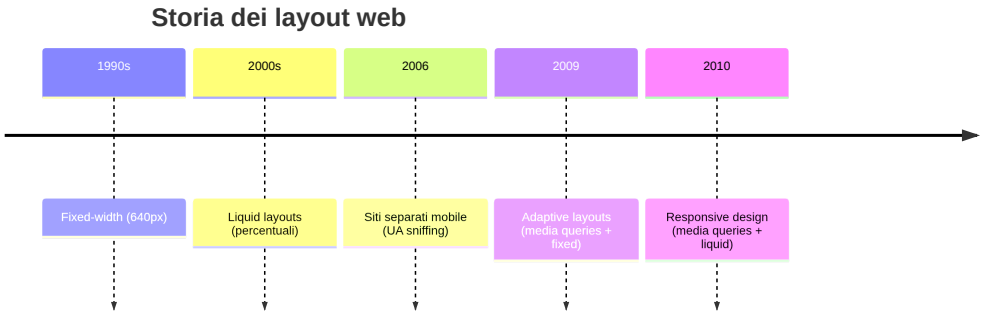
- Con le media query (~2009), i designer passavano tra un **piccolo numero di design fixed-width** usando media query: è l'**adaptive design**.
- L'utente vede il layout più vicino alla dimensione del proprio browser; per le dimensioni intermedie il design non è ottimale.

```css
@media (max-width: 800px) { main { width: 400px; } }
@media (min-width: 800px) { main { width: 500px; } }
```

### Responsive layouts

- **Adaptive** = media queries + fixed-width; **responsive** = media queries + liquid layouts (termine coniato da Ethan Marcotte, 2010).
- Il responsive design è caratterizzato da: **container fluidi**, **media fluidi**, **media queries**.
- Layout e immagini di un sito responsive devono apparire ben resi su qualsiasi dispositivo.

```css
body { display: flex; flex-wrap: wrap; }
@media (min-width: 768px) { main { width: 70%; } aside { width: 20%; } }
img { width: 20%; min-width: 100px; }
```




### Viewport meta tag

- I primi browser mobili assumevano che i siti fossero progettati per uno schermo di **980px**.
- Rendering in un **virtual viewport** di 980px, poi **scalatura** della pagina alla larghezza reale dello schermo.




- Il meccanismo aiuta i siti non ottimizzati per mobile, ma **non funziona** per i siti mobile-optimized: le media query che scattano a 640px non si attivano mai a 980px.
- Il **meta tag viewport** permette di controllare il meccanismo; le pagine mobile-optimized dovrebbero includere in `<head>`:

```html
<meta name="viewport" content="width=device-width, initial-scale=1">
```

- `width=device-width`: assumere che la larghezza di progettazione sia la larghezza del

device.
- `initial-scale=1`: nessuna scalatura.

Capitolo 7

# Lezione 07: Modern, Adaptive CSS

Estensione dei concetti già visti (selettori, cascata, box model, normal flow, posizionamento, Flexbox, Grid, media query) tramite nuove funzionalità del CSS moderno.

## Pseudo-classi funzionali

Info

**Contesto**

I selettori semplici possono combinarsi in relazioni complesse, ma le liste di selettori ripetitive allungano i fogli di stile (meno efficienti da caricare) e rendono le modifiche più soggette a errori. Strumenti come **Sass** (pre/post-processore) si sono diffusi proprio per organizzare i fogli di stile; il CSS moderno ha incorporato direttamente nel linguaggio capacità analoghe.

Definition

**Pseudo-classi funzionali**

Pseudo-classi che accettano selettori come argomenti e consentono di scrivere selettori in modo più conciso. - `:is()` raggruppa selettori alternativi. - `:where()` crea default facilmente sovrascrivibili. - `:not()` esclude gli elementi che corrispondono. - `:has()` seleziona elementi in base a elementi correlati.

### `:is()` (matches-any)

- `:is()` corrisponde a un elemento se uno dei suoi selettori argomento corrisponde; è utile quando in un selettore complesso cambia solo una parte.

```css
:is(header, main, footer) a em { color: red; }
button:is(:hover, :focus, :active) { color: red; }
```

### `:where()`

- `:where()` seleziona come `:is()`, ma contribuisce **specificità zero** alla cascata, per sé e per tutti i suoi argomenti.
- La conseguenza è che gli stili risultanti sono volutamente facili da sovrascrivere (utile per regole di default).

### `:not()`

- `:not()` corrisponde agli elementi che non corrispondono a nessuno dei suoi argomenti.

### `:has()`

- `:has()` corrisponde a elementi in base ai loro discendenti o elementi correlati, anche fratelli successivi.

```css
ol:has(.active) { background-color: lightgray; }
li:has(+ .selected) { color: red; }
```

#### Specificità delle pseudo-classi funzionali

- Le pseudo-classi funzionali **non contano come pseudo-classi** nel calcolo della specificità (non entrano in B come `:focus` o `:hover`).
- `:where()` e il suo contenuto contribuiscono sempre **zero specificità**.
- Per `:is()`, `:not()` e `:has()` la specificità è quella dell'argomento più specifico.

## Custom properties (variabili CSS)

- I valori ripetuti nel CSS sono difficili da mantenere: se un valore cambia, va modificato in ogni occorrenza.

Definition

**Custom properties**

Entità definite dagli autori CSS che rappresentano valori specifici riutilizzabili; si definiscono con la sintassi delle proprietà personalizzate (o con l'at-rule `@property`) e si recuperano con `var()`.

```css
:root {
  --primary-color: #663399;
}
h1 { color: var(--primary-color); }
```

#### Ereditarietà

- Le custom properties **partecipano alla cascata**, sono **normalmente ereditate** e possono essere **sovrascritte**.
- La pseudo-classe `:root` corrisponde alla radice dell'albero del documento e permette di definire custom property «globali»; un elemento antenato può poi ridefinire la stessa proprietà, e i discendenti ereditano il nuovo valore.

```css
:root { --accent: #cfa2fb; }
button { background: var(--accent); }
.warning { --accent: orangered; }
```

### Fallback

- **Fallback**: `var()` accetta un valore di riserva come secondo argomento, usato se la custom property non è definita.

```css
.notice { color: var(--notice-color, darkred); }
```

- I fallback sono utili quando un componente ammette **personalizzazione opzionale**.

### At-rule `@property`

- `@property` permette di personalizzare il comportamento di ereditarietà di una custom property, dichiarandone sintassi, ereditarietà e valore iniziale.

```css
@property --accent {
  syntax: "<color>";
  inherits: false;
  initial-value: teal;
}
```

## Valori calcolati

### `calc()`

- `calc()` combina valori e unità compatibili con gli operatori aritmetici `+`, `-`, `*`, `/`.

```css
.item { width: calc(100% - 20px); margin: auto; }
```

### `min()` e `max()`

- `min()` e `max()` selezionano rispettivamente il più piccolo e il più grande dei loro argomenti.
- Con `width: min(100px, calc(100% - 2rem))` l'elemento non supera mai 100px, si restringe quando lo spazio disponibile è minore e conserva almeno 1rem per lato.
- Con `font-size: max(1rem, 10vw)` la dimensione del font non scende mai sotto 1rem e può crescere su viewport più larghi.

### `clamp()`

- `clamp()` vincola un valore tra un minimo e un massimo: il valore cresce fluidamente ma non supera mai i limiti.

# Unità viewport su dispositivi mobili

- **100vh** rappresenta l'altezza del viewport; su desktop si basa sulla dimensione corrente della finestra, ma su mobile i controlli del browser (es. barra degli indirizzi) possono apparire e scomparire, complicando il calcolo.

Info

**Unità viewport moderne**

- **svh** (small viewport height): altezza del viewport calcolata con la barra di navigazione visibile. - **lvh** (large viewport height): calcolata sull'altezza massima possibile del viewport (come vh). - **dvh** (dynamic viewport height): rappresenta dinamicamente l'altezza corrente del viewport; con i controlli visualizzati corrisponde a lvh meno l'altezza dei controlli (svh), senza controlli coincide con lvh.

# Pattern di layout moderni

### gap

- La proprietà **gap** si usa sui container **flex** e **grid** per aggiungere spazio tra i figli.
- A differenza dei margini sui singoli figli, gap aggiunge spazio **solo tra gli elementi** (non ai bordi del container).

### Shorthand flex

- Lo shorthand **flex** controlla quanto un item cresce (**flex-grow**), quanto si restringe (**flex-shrink**) e la sua dimensione base iniziale (**flex-basis**).

CSS

```
.item { flex: 1 1 100px; } /* equivale a flex-grow:1; flex-shrink:1;
    flex-basis:100px */
```

# Container queries

- Le **media query** osservano il viewport, ma lo stesso componente può trovarsi in container di dimensioni diverse (area principale oppure sidebar/colonna di griglia più piccola): in un container piccolo il componente dovrebbe comportarsi come in un viewport piccolo, anche se il viewport è grande.

Definition

**Container queries**

Regole che permettono di stilare un componente in base alla dimensione di un antenato; richiedono la dichiarazione della proprietà container-type sull'antenato e l'uso dell'at-rule @container per la query.

```css
aside { container-type: inline-size; }
@container (max-width: 300px) {
  .card { background-color: lightgreen; }
  .card :is(h2, p) { font-size: 1em; }
}
```

# Candidati immagine responsive

- Il CSS determina quanto grande viene mostrata un'immagine, ma scaricare un'immagine ad alta risoluzione e scalarla su schermi piccoli è inefficiente.
- L'**HTML** può fornire più file per la stessa immagine: il browser seleziona il migliore candidato in base al dispositivo, scaricando solo la versione necessaria, tramite l'attributo `srcset` o l'elemento `<picture>`.

## `srcset`

- L'attributo `src` funge da fallback (quando `srcset` non è supportato); `srcset` specifica una lista di nomi di file con **descrittori**:
  - **descrittori di larghezza** (numero seguito da `w`): dimensione intrinseca dell'immagine;
  - **descrittori di densità di pixel** (numero seguito da `x`): densità di pixel del dispositivo rispetto alla densità standard.

```html
<img src="fu-tzu-400.jpg"
     srcset="fu-tzu-400.jpg 400w, fu-tzu-800.jpg 800w, fu-tzu-1200.jpg
    1200w"
     alt="Portrait of the great master Fu-Tzu">
```

## `<picture>` e art direction

**Definition**

**Art direction**

Processo di selezione del media migliore per condizioni specifiche: non solo diverse risoluzioni della stessa immagine, ma anche contenuti diversi (es. un grafico con meno dettagli su mobile e completo su desktop).

```html
<picture>
  <source media="(max-width: 200px)" srcset="fu-tzu-400.png">
  <source media="(max-width: 400px)" srcset="fu-tzu-800.png">
  <img src="fu-tzu-1200.png" alt="Portrait of the great master Fu-Tzu">
</picture>
```

## Media query e preferenze utente

- Le media query possono testare più delle dimensioni del viewport: riflettono anche le **preferenze dell'utente**, normalmente originate dalle impostazioni del browser o del sistema operativo:
  - **schema di colori preferito**: `@media (prefers-color-scheme: dark)`;
  - **movimento ridotto**: `@media (prefers-reduced-motion: reduced)`;
  - **contrasto aumentato o ridotto**: `@media (prefers-contrast: more)`;
  - **trasparenza ridotta**: `@media (prefers-reduced-transparency: reduced)`.

CSS

```css
:root { --bg-color: white; --text-color: indigo; }
@media (prefers-color-scheme: dark) {
  :root { --bg-color: indigo; --text-color: white; }
}
```

## Query `@supports`

- `@supports` applica gli stili solo quando certe coppie proprietà/valore sono supportate; la regola base funge da **fallback** e il layout avanzato (es. griglia) è applicato solo se il browser lo supporta.

CSS

```css
.cards { display: block; }
@supports (display: grid) {
  .cards { display: grid; grid-template-columns: 1fr 1fr; }
}
```

## Nesting nativo del CSS

- Nel CSS moderno i selettori correlati possono essere **nidificati** all'interno di altre regole, evitando di ripetere il selettore padre.

CSS

```css
.card {
  background-color: lightblue;
  :is(h2, p) { font-size: 1.25em; }
}
```

### Selettore di nesting: &

- Il selettore `&` in un selettore nidificato si riferisce esplicitamente al selettore della regola padre.

```css
.card {
  background: white;
  &:hover { background: lavender; }
  & > h2 { color: rebeccapurple; }
}
```

Warning

**Prudenza con il nesting**

Mantenere il nesting poco profondo: un eccesso di nidificazione rende i selettori e il comportamento della cascata più difficili da capire.

# Parte V

# JavaScript

Capitolo 8

# Lezione 8 JavaScript

## Il linguaggio

Definition

**JavaScript**

**Linguaggio di programmazione** scripting di alto livello, debolmente tipato (loosely-typed). Introdotto dal browser Netscape per rendere dinamici i documenti HTML: i programmi (script) sono interpretati come puro testo dal **JS Engine**. Implementa la specifica **ECMAScript**.

- Oggi è usato anche fuori dalle pagine web: backend, applicazioni desktop e mobile, scripting generico.
- Supporta i paradigmi **imperativo**, **funzionale** e **orientato agli oggetti**, con feature dedicate alla programmazione **asincrona**.
- Le pagine HTML+CSS sono intrinsecamente **statiche**: JavaScript permette al contenuto di cambiare durante la navigazione.

### Includere JavaScript in una pagina

- **JavaScript interno**: codice dentro un elemento `<script>` in `<head>` o `<body>`.
- **JavaScript esterno**: file con estensione `.js` referenziato tramite `<script src="...">`.

HTML

```
<script> console.log("Hello World!"); </script>
<script src="script.js"></script>
```

### Strict mode

- Fino al 2009 (ECMAScript 5) il linguaggio è evoluto senza **breaking changes**; ES5 ne ha introdotti alcuni, disattivati di default per retro-compatibilità.
- Si abilitano con la direttiva `"use strict"` alla prima riga dello script. Nel corso si usa sempre la versione moderna «strict».

JavaScript

```
"use strict";
/* JavaScript code here */
```

# Sintassi di base

- Un programma è una **sequenza di statement**, composti da valori, operatori, espressioni, keyword e commenti.
- La sintassi è simile a Java e C ma più permissiva; gli statement sono delimitati da newline e/o **punto e virgola** (uso preferito come buona pratica).
- Con `typeof` si ispeziona il tipo di un valore: `undefined`, `number`, `string`, `boolean`.

### Operatori

| Operatore | Descrizione |
|---|---|
| `=` | Assegnamento |
| `+ - * /` | Somma, sottrazione, moltiplicazione, divisione |
| `**` | Elevamento a potenza (da ECMAScript 2016) |
| `%` | Modulo (resto della divisione) |
| `++ --` | Incremento, decremento |

| Operatore | Descrizione |
|---|---|
| `==` | uguaglianza debole (con conversione di tipo) |
| `===` | uguaglianza stretta (stesso valore e stesso tipo) |
| `!=` | diverso (debole) |
| `!==` | diverso valore o diverso tipo |
| `> < >= <=` | confronti |
| `?` | operatore ternario |

Info

**Uguaglianza debole vs stretta**

- `==` converte gli operandi di tipo diverso prima di confrontarli: `"1" == 1` e `true == "1"` sono `true`. - `===` non converte: se i tipi differiscono restituisce subito `false` (`"1" === 1` è `false`).

### Controllo di flusso

`if`, `if-else`, `for`, `while`, `do-while` e `switch` hanno stessa sintassi e semantica di Java; anche `continue` e `break` funzionano allo stesso modo.

# Variabili e scope

### Ciclo di vita

- **Dichiarazione**: il nome della variabile è legato allo scope corrente.
- **Inizializzazione**: alla variabile viene assegnato un valore (di default `undefined`).
- **Uso**: la variabile può essere referenziata.

Definition

**Temporal Dead Zone (TDZ)**

Intervallo tra la dichiarazione e l'inizializzazione in cui la variabile è **inaccessibile**: referenziarla solleva un `ReferenceError`.

### Dichiarazione

- `let`: variabili «standard», con **scope a livello di blocco** (il blocco `{} più vicino`).
- `const`: costanti con scope di blocco; non possono essere riassegnate ( tentativo → `TypeError`).
- Le variabili sono accessibili solo dopo l'esecuzione della riga in cui sono dichiarate; nello stesso blocco non si può ridichiarare lo stesso nome.

JavaScript

```
let bool;
bool = true;
if (bool) {
    let msg = "Hello"; // visibile solo dentro il blocco
    console.log(msg);
}
```

- Una variabile dichiarata in un blocco interno **oscura** (shadowing) l'omonima dello scope esterno.

### Hoisting

Definition

**Hoisting**

Comportamento predefinito per cui le dichiarazioni di variabili (e funzioni) sono implicitamente spostate all'inizio del loro scope. Con `let/const` è sollevata solo la **dichiarazione**, non l'inizializzazione, che avviene alla riga originaria: l'uso precedente cade nella TDZ.

JavaScript

```
{
    console.log(x); // ReferenceError: can't access 'x' before
    initialization
    let x = 1;
}
```

### `var` e dichiarazione implicita (pre-ECMAScript 6)

Warning

**var è deprecato**

- `var` ha scope di funzione/globale (nessuno scope di blocco) ed è hoistato **con inizializzazione a `undefined`**: usare la variabile prima della riga di dichiarazione stampa `undefined` invece di un errore, e la variabile resta visibile fuori dal blocco. - **Dichiarazione implicita** (assegnamento senza keyword): crea una **variabile globale**, pratica error-prone, vietata in strict mode (`ReferenceError`).

JavaScript

```
if (true) {
    console.log(n); // undefined (non errore!)
    var n = 1;
}
console.log(n); // 1
```

## Scope

- **Scope globale**, **scope di funzione**, **scope di modulo**; le variabili `let`/`const` aggiungono lo **scope di blocco**.

## Funzioni

JavaScript

```
function greet(name, message = "Hello") {
    console.log(`${message} ${name}`);
}
```

- La **dichiarazione di funzione** vale nello scope in cui avviene; le dichiarazioni di funzione sono **hoistate** e quindi richiamabili prima della riga che le definisce.
- Se un argomento non è passato, il parametro è `undefined`; i parametri possono avere **valori di default**.
- I **template literals** (backtick) permettono l'interpolazione `${...}`.

### Scope interno e visibilità

- Una funzione crea il proprio scope: le variabili e funzioni dichiarate al suo interno non sono visibili all'esterno.
- Una funzione può accedere alle variabili degli scope esterni (**closure**), purché non mascherate nel proprio scope.

### Function expressions

- Le funzioni sono **valori**: possono essere assegnate a variabili (**function expression**), anche con sintassi **arrow function**.
- Si applicano le regole di hoisting delle variabili: la funzione non è usabile prima della riga di assegnazione (TDZ), a differenza delle dichiarazioni.

```JavaScript
let greet = function(name) { console.log(`Hello ${name}!`); };
let salute = (name) => { console.log(`Howdy ${name}!`); };
```

### Funzioni annidate e closure

Definition

**Funzione annidata**

Funzione creata dentro un'altra funzione. Può essere **restituita** e usata fuori dalla funzione originaria e, ovunque venga usata, conserva l'accesso al contesto esterno (**closure**) della funzione che l'ha creata.

```JavaScript
function getGreeter(message) {
    let sep = ",";
    return function(name) {
        console.log(`${message}${sep} ${name}!`);
    }
}
let helloGreeter = getGreeter("Hello"); // "cattura" message e sep
```

## Oggetti

Definition

**Oggetto**

Contenitore di dati **chiave: valore**. Si crea con la sintassi **object constructor** (`new Object()`) o, più comunemente, **object literal** (`{}`). Una **proprietà** ha una chiave (nome) prima di `:` e un valore a destra; le proprietà sono separate da virgole.

### Proprietà

- **Accesso con dot notation**: `pet.name`; proprietà inesistente → `undefined`.
- **Aggiunta/cancellazione**: `pet.nickname = "..."` aggiunge, `delete pet.age` rimuove.
- **Square brackets notation**: necessaria per chiavi con spazi o espressioni: `dog["is a good boy"]`, `dog["is the " + "best boy"] = true`.

### Riferimenti

- Una variabile assegnata a un oggetto memorizza un **riferimento**, non l'oggetto: due variabili che puntano allo stesso oggetto sono uguali sia con `==` che con `===` (condividono lo stesso valore di riferimento).

### Cloning

- **Copia per riferimento**: assegnare un oggetto a un'altra variabile non copia nulla; le modifiche sono visibili da entrambe le variabili.
- **Shallow clone**: copia proprietà per proprietà (es. con un ciclo `for...in`); gli oggetti annidati restano **condivisi** (modificare `copy.owner.name` modifica anche l'originale).
- **Deep clone**: richiede **ricorsione** — se il valore di una proprietà è un oggetto, si clona ricorsivamente.
- Alternative built-in: `Object.assign()` (shallow), `structuredClone()` (deep).

JavaScript

```
function clone(object) {
    let copy = {};
    for (let key in object) {
        copy[key] = (typeof object[key] === "object") ? clone(object[key
    ]) : object[key];
    }
    return copy;
}
```

### Metodi

- Una proprietà il cui valore è una funzione è un **metodo**; definibili in assegnamento, con sintassi classica `greet: function() {...}` o **shorthand** `greet() {...}`.

### `this`

Definition

**this**

Riferimento valutato **a run-time** in base al contesto: dentro un metodo, punta all'oggetto che contiene il metodo, permettendogli di accedere alle altre proprietà dello stesso oggetto.

JavaScript

```
let john = { name: "John", greet() { console.log(`Hi, I'm ${this.name}!`)
    ; } };
```

- La stessa funzione assegnata come metodo a oggetti diversi vede ogni volta l'oggetto chiamante tramite `this`.
- **Le arrow function non hanno `this`**: il `this` al loro interno è preso dal contesto esterno, quindi non riflette l'oggetto chiamante.

### Costruttori

**Definition**

**Funzione costruttore**

Funzione regolare invocata con la keyword `new` per creare molti oggetti simili. Convenzioni: nome con iniziale maiuscola e invocazione solo con `new`.

Quando una funzione è eseguita con `new`: 1) si crea un nuovo oggetto vuoto assegnato a `this`; 2) viene eseguito il corpo, che tipicamente modifica `this`; 3) `this` è restituito.

JavaScript

```
function Pet(name, species) {
    this.name = name;
    this.species = species;
}
let rick = new Pet("Richard", "Lizard");
```

### Optional chaining

- Accedere una proprietà di una proprietà `undefined` solleva un errore; l'operatore `?.` «cortocircuita»: se la parte sinistra è `undefined` restituisce subito `undefined` invece di un errore.
- Sostituisce il controllo esplicito `pet.owner ? pet.owner.name : undefined`.
- Varianti: `?.()` per chiamare una funzione che potrebbe non esistere (`mike.greet?.()` → `undefined`), `?.[chiave]` con la notazione a parentesi.

### Configurazione delle proprietà

- Ogni proprietà ha tre **flag**: **Writable** (il valore può cambiare), **Enumerable** (la proprietà compare nei cicli), **Configurable** (la proprietà può essere cancellata e i flag modificati).
- Con la creazione «tradizionale» tutti i flag sono `true`; con `Object.defineProperty(obj, key, { value, writable, enumerable, configurable })` si configurano esplicitamente (es. proprietà read-only → `TypeError` alla riassegnazione o alla cancellazione se non configurabile).

### Getter e setter

**Info**

**Proprietà dati vs accessorie**

- **Proprietà dati**: memorizzano un valore. - **Proprietà accessorie**: funzioni eseguite quando la proprietà è letta (**getter**, keyword `get`) o assegnata (**setter**, keyword `set`).

JavaScript

```
let p = {
    first: "John", last: "Smith",
    get fullName() { return `${this.first} ${this.last}`; },
    set fullName(name) { [this.first, this.last] = name.split(" "); }
};
```

```
6  p.fullName; // si accede come proprietà, senza ()
```

- Getter e setter si usano **come proprietà** (senza `()`): dall'esterno non c'è differenza tra proprietà dati e accessorie.

# Lezione 9 JavaScript

## Prototipi ed ereditarietà

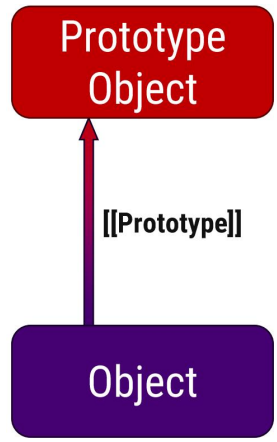
### Il prototype

Definition

**Ereditarietà prototipale**

Ogni oggetto possiede una proprietà nascosta **`[[Prototype]]`**, che è `null` oppure riferimento a un altro oggetto. Quando si accede a una proprietà mancante, JavaScript la cerca risalendo la **catena dei prototipi**.

- **`__proto__`** è un getter/setter per la proprietà `[[Prototype]]`; in JavaScript moderno si preferiscono `Object.getPrototypeOf()` e `Object.setPrototypeOf()`.
- I prototipi sono usati **solo in lettura**: le operazioni di **scrittura e cancellazione** agiscono direttamente sull'oggetto, non sul prototipo.



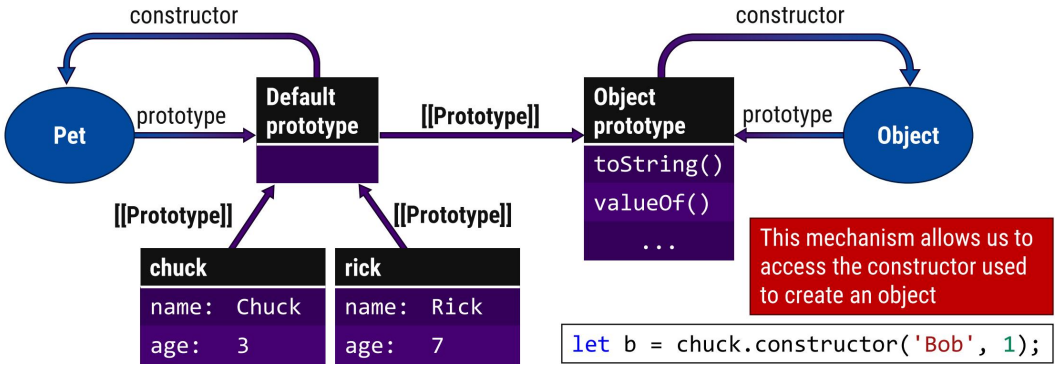

JavaScript

```
let pet = { legs: 4 };
let cat = { name: "Garfield" };
Object.setPrototypeOf(cat, pet);
cat.legs;      // 4 (dalla catena)
snake.legs = 0; // snake ora ha una proprietà propria legs
```

L'API standard moderna raccomandata rispetto a **proto**, ma va usata con cautela perché fa crollare le ottimizzazioni del motore JS. ### Costruttori e prototype

- Con una **funzione costruttore** invocata con `new`, l'operatore usa `Constructor.prototype` per impostare il `[[Prototype]]` del nuovo oggetto.
- Se il costruttore non ha una proprietà `prototype`, viene usato un **prototipo di default**: un oggetto con la proprietà `constructor` che punta alla funzione costruttore stessa (`Pet.prototype = { constructor: Pet }`).
- Questo meccanismo consente di risalire al costruttore tramite `obj.constructor`.

```JavaScript
function Duck(name) { this.name = name; }
Duck.prototype = { eats: true };
let chuck = new Duck("Chuck");
chuck.eats; // true, via prototype
```




### Metodi: due alternative

Esistono due modi per dotare di metodi gli oggetti creati da un costruttore:

- Assegnare il metodo a `this` dentro il costruttore: ogni oggetto ha una **copia propria** del metodo.
- Assegnarlo a `Constructor.prototype`: il metodo è **condiviso** da tutti gli oggetti tramite la catena dei prototipi.

```JavaScript
Pet.prototype.describe = describe; // metodo condiviso sul prototype
```

## Strutture dati

### Array

- Un **array** memorizza una sequenza ordinata di valori, dichiarabile con la sintassi letterale `[...]` o con il costruttore `new Array(...)`.
- Gli array sono **indicizzati da 0**; l'accesso in lettura/scrittura usa la notazione a parentesi quadre.
- La proprietà **`length`** non è il conteggio dei valori: contiene l'indice massimo + 1, ed è **scrivibile** (assegnare un indice alto la aumenta, ridurla tronca l'array).
- Un array può contenere **tipi eterogenei** (stringhe, oggetti, numeri, funzioni).
- Gli array possono contenere altri array, formando **array multidimensionali** (es. matrici, accessibili con `matrix[i][j]`).

```JavaScript
let a = [1, 2, 3];
a[999] = 998;   // a.length diventa 1000
a.length = 3;   // tronca l'array
```

### Metodi di aggiunta/rimozione

- **`push()`**: aggiunge in coda; **`unshift()`**: aggiunge in testa.
- **`pop()`**: estrae dalla coda; **`shift()`**: estrae dalla testa.

### Iterazione

- **`for` classico** sugli indici e **`for..of`** percorrono tutti gli elementi (inclusi eventuali *buchi* come `undefined`).
- **`forEach()`** riceve una callback `(value, index, array)`.
- **`for..in`** è sconsigliato sugli array: è 10–100 volte più lento e può includere altre proprietà enumerabili.

## Destructuring

Definition

**Destructuring assignment**

Sintassi che permette di **spacchettare** un array in variabili: `let [x, y] = ...`. Gli elementi rimanenti sono scartati, oppure raccolti con l'operatore **rest** `...`.

```JavaScript
let [a, b, ...rest] = [1, 2, 3, 4, 5]; // rest = [3, 4, 5]
```

- Una sintassi simile vale per i **parametri di funzione**: `function greet(msg, ...names)` raccoglie gli argomenti extra nell'array `names`.

### Iterabili e Symbol.iterator

- Il **`for..of`** funziona con oggetti **iterabili**: array e stringhe lo sono.
- Per rendere iterabile un oggetto custom occorre implementare il metodo speciale **`Symbol.iterator`**.

Info

**Protocollo di iterazione**

- All'avvio del ciclo, `for..of` chiama `Symbol.iterator` una volta (in sua assenza lancia un `TypeError`). - Da quel momento lavora solo con l'**iterator** restituito. - Per ogni valore chiama `next()`, che restituisce `{ done: boolean, value: any }`: `done: true` termina il ciclo, altrimenti `value` è il prossimo valore.

```JavaScript
range[Symbol.iterator] = function() {
    return { current: this.min, max: this.max, step: this.step,
        next() {
            let n = { done: false, value: this.current };
            if (n.value > this.max) n.done = true;
```

```
            this.current += this.step;
            return n;
        } };
};
```

- È possibile usare l'iteratore **esplicitamente**: chiamare `next()` in un ciclo `while` fino a `done` replica il comportamento di `for..of`.

### Map

**Definition**

**Map**

Collezione di **coppie chiave-valore** a differenza degli oggetti, ammette **chiavi di qualsiasi tipo** (non solo stringhe).

| Metodo / proprietà | Descrizione |
|---|---|
| `new Map()` | crea la map |
| `map.set(key, value)` | memorizza il valore associato alla chiave |
| `map.get(key)` | restituisce il valore, o `undefined` se assente |
| `map.has(key)` | `true` se la chiave esiste |
| `map.delete(key)` | rimuove la coppia |
| `map.clear()` | svuota la map |
| `map.size` | numero di elementi |

- In un oggetto le chiavi sono **coercite a stringhe**: `{1: "Num"}` e `{"1": ...}` collidono; in una `Map` i tipi restano distinti.

#### Iterazione sulle Map

- **`map.keys()`**, **`map.values()`** e **`map.entries()`** restituiscono iterabili su chiavi, valori e coppie.
- Iterare direttamente la map (`for (let [key, value] of map)`) equivale a usare `entries()`.

### Set

**Definition**

**Set**

Struttura per collezioni di **valori unici**, senza ripetizioni.

| Metodo / proprietà | Descrizione |
|---|---|
| `new Set([iterable])` | crea il Set, copiando i valori di un iterabile |
| `set.add(value)` | aggiunge il valore e restituisce il set |
| `set.delete(value)` | elimina il valore; `true` se esisteva |
| `set.has(value)` | `true` se il valore è presente |

| Metodo / proprietà | Descrizione |
|---|---|
| `set.clear()` | svuota il set |
| `set.size` | numero di elementi |

# Classi

### La sintassi class

- Una **classe** è un template per creare oggetti; `new Pet(...)` crea l'oggetto e invoca `constructor()` con gli argomenti, che imposta le proprietà sull'oggetto nuovo, poi lo restituisce.

```JavaScript
class Pet {
    constructor(name) { this.name = name; }
    greet() { /* ... */ }
}
let pet = new Pet("Chuck");
```

**Info**

**Cosa fa davvero class**

Una classe **non è una nuova entità del linguaggio**: `typeof Pet` è `"function"`. 1. Crea una funzione costruttore il cui codice è preso da `constructor()`. 2. Memorizza gli altri metodi in `Pet.prototype`.

- Come gli oggetti, le classi supportano **proprietà** di istanza e **getter/setter** (`get`), e possono definire metodi speciali come `[Symbol.iterator]`.

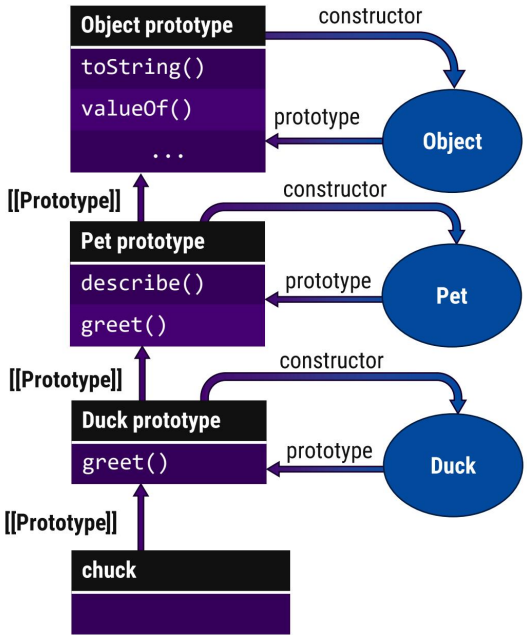
```JavaScript
class Duck {
    species = "Duck";           // proprietà
    get fullDescription() { return `${this.name} the ${this.species}`; }
}
```

### Ereditarietà: extends

- Una classe può ereditare da un'altra con la keyword **`extends`**; i metodi della classe base restano accessibili, quelli ridefiniti vengono **sovrascritti** (override).

```JavaScript
class Duck extends Pet {
    greet() { console.log("Quack!"); } // override
}
```

- Internamente, `extends` è implementato tramite la proprietà **`[[Prototype]]`**: il `<prototype>` di `Duck` punta a `Pet`, che a sua volta punta a `Object`.




# Gestione degli errori

### Errori e try/catch/finally

- A runtime possono verificarsi **errori** (errori del programmatore, input inattesi): normalmente interrompono lo script con un messaggio in console.
- Il costrutto **`try/catch/finally`** gestisce gli errori: `try` contiene il codice a rischio, `catch` riceve l'oggetto `error` (con `name` e `message`), `finally` è eseguito sempre.

```JavaScript
try {
    x = 1; // errore
} catch (error) {
    console.log(error.name, error.message);
} finally {
    console.log("done");
}
```

- In JavaScript c'è **al massimo un blocco `catch`** (`try/finally` da solo è ammesso); i diversi tipi di errore si distinguono dentro il blocco, ad esempio con `instanceof`.
- L'oggetto `error` è gestito tipicamente verificando `error instanceof ReferenceError`.

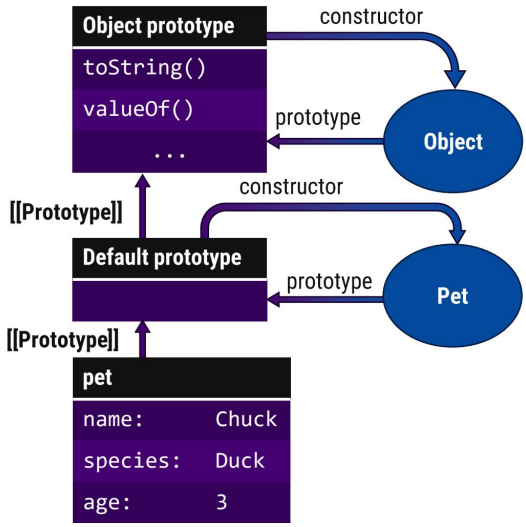
### instanceof

**Definition**

**Operatore instanceof**

Verifica se la proprietà `prototype` di un costruttore **compare nella catena dei prototipi** di un oggetto.

```JavaScript
pet instanceof Pet;    // true
pet instanceof Object; // true
```




### throw

- Gli errori **si propagano verso l'alto** nella call tree finché non sono catturati o si raggiunge la cima (in tal caso lo script si interrompe).
- Si possono lanciare errori con la keyword **`throw`**, tipicamente `throw new Error("...")`.
- È possibile definire **classi di errore custom** estendendo `Error` e sovrascrivendo `this.name` nel costruttore (chiamando `super(message)`).

```JavaScript
class MyError extends Error {
    constructor(message = "Whoops") { super(message); this.name = "
    MyError"; }
}
```

# Moduli

## Modularità

Definition

### Modulo

Un modulo è semplicemente un **file JavaScript**; i moduli si caricano a vicenda tramite le direttive `export` ed `import`, per scambiare funzionalità.

- La modularità ** migliora la manutenibilità** e promuove la separazione delle *concerns*.
- Aiuta a prevenire **conflitti di naming**: identificatori uguali in moduli diversi non collidono.

Warning

### Perché gli script semplici non bastano

Includere più `<script>` nella pagina mette tutto in un unico scope globale: due `let msg` con lo stesso nome provocano un `SyntaxError: redeclaration of let msg`. Il riuso del codice richiederebbe di rinominare gli identificatori, vanificando lo scopo.

## export e import

- `export` etichetta le variabili e le funzioni accessibili dall'esterno del modulo; tutto il resto rimane **privato**.
- `import` importa funzionalità specifiche da altri moduli (solo se esportate), specificandone l'URL; si può rinominare con `import { greet as greetPet }`.
- Nell'HTML basta includere lo script principale come modulo: `<script type="module" src="./script-module.js"></script>`.

JavaScript

```
// pet-module.js
export function Pet(name, age) { this.name = name; this.age = age; }
let msg = "Hello"; // privato, non esportato

// script-module.js
import { Pet, greet as greetPet } from './pet-module.js';
```

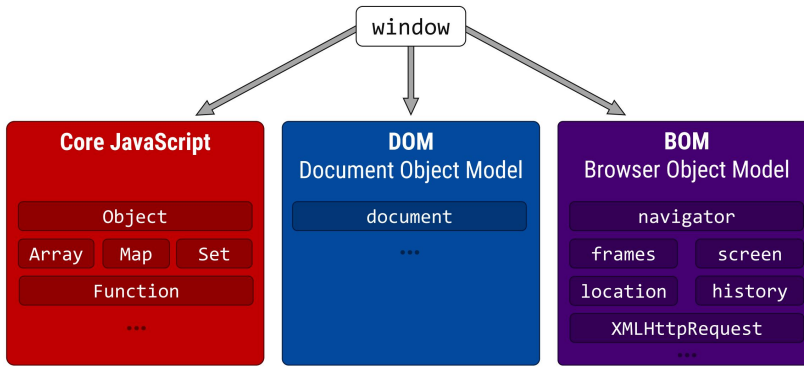
# Lezione 11 — JavaScript in a Browser Environment

## L'ambiente browser

### Host environment

JavaScript può girare in diversi **host environment** (ambienti host):

- **Browser web**: per cui JavaScript è stato originariamente creato.
- **Web server**: es. tramite il runtime **Node.js**.
- Ogni host environment fornisce i propri **oggetti** e **funzioni**, in aggiunta a quelli inclusi nel **core del linguaggio**.




### window

L'ambiente browser ha un oggetto «radice» chiamato `window`.

- È l'**oggetto globale** per il codice JavaScript.
- Rappresenta la finestra del browser e fornisce **metodi per controllarla**.
- Le funzioni e le variabili globali sono **proprietà di `window`**.

JavaScript

```
let msg = "Hello";
function greet(name) { console.log(`${msg}, ${name}!`); }

greet("Web Technologies");     // richiamata come funzione globale
window.greet("window object"); // la stessa funzione, come proprietà di
    window
```

### DOM

**Definition**

**DOM (Document Object Model)**

Rappresenta il **contenuto del documento corrente**. L'oggetto `document` è il punto d'ingresso principale alla pagina web e fornisce modi per **accedere** e **manipolare** i contenuti.

### BOM

**Definition**

**BOM (Browser Object Model)**

Insieme di oggetti aggiuntivi forniti dal browser per lavorare **sul browser stesso**, non sui contenuti del documento.

Gli oggetti principali del BOM sono i seguenti:

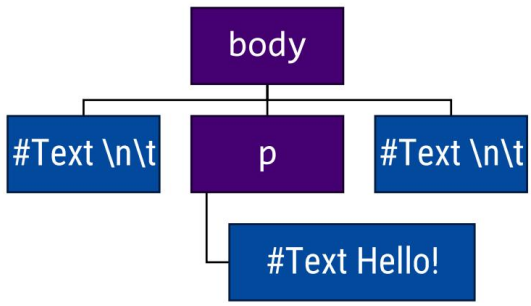
- `location`: contiene informazioni sull'URL del documento corrente (`location.href`).
- `screen`: informazioni sul display usato dall'utente (`screen.availWidth`).
- `navigator`: dettagli aggiuntivi su browser e piattaforma (`navigator.userAgent`).
- `history`: rappresenta la cronologia di navigazione (`history.back()`, come il pulsante «back» del browser).

# Il DOM

### Struttura ad albero

Un documento HTML può essere visto come un **albero** (con discendenti, figli, fratelli): il **DOM formalizza** questa intuizione.

- Ogni **tag HTML è un oggetto** nel DOM.
- I tag annidati sono **figli** del tag che li racchiude.
- Il contenuto testuale di un tag è un oggetto (`#text`) a sua volta **figlio del tag**.
- Gli **attributi** di un tag sono accessibili come **proprietà** dell'oggetto corrispondente.
- Il DOM reale include anche **spazi e a-capo** tra i tag come nodi `#text`; nelle rappresentazioni si omettono per brevità.




I **commenti HTML** non sono mostrati dal browser, ma vengono comunque analizzati e aggiunti al DOM come speciali **nodi commento**.

### Tipi di nodo

La specifica DOM definisce **12 tipi di nodo**; tipicamente si lavora con quattro tipi di nodo:

- **Document node**: è la radice e il punto d'ingresso del DOM.
- **Element node**: rappresenta un elemento HTML.
- **Text node**: rappresenta il contenuto testuale.
- **Comment node**: rappresenta un commento.

### Trovare elementi

L'oggetto `document` fornisce diversi **metodi per selezionare elementi** da un documento HTML.

- **`querySelector`** e **`querySelectorAll`** sono i metodi più versatili e ricevono in input un **selettore CSS**.
- `querySelector` restituisce solo il **primo elemento** corrispondente al selettore; `querySelectorAll` restituisce la **lista completa** delle corrispondenze.
- Tutti i metodi restituiscono **`null`** quando non trovano elementi.

```JavaScript
document.getElementById("a");            // elemento per attributo id
document.getElementsByClassName("foo");  // HTMLCollection
document.getElementsByName("c");        // NodeList
document.querySelector("ul > li");       // primo <li> che corrisponde
document.querySelectorAll("ul > li");    // tutti i <li> che
    corrispondono
```

**Definition**

**HTMLCollection («live collection»)**

Tutti i metodi della famiglia **`getElementsBy*`** restituiscono una **HTMLCollection**: una collezione aggiornata automaticamente, che riflette sempre lo stato corrente del documento.

```JavaScript
let links = document.getElementsByTagName("a"); // HTMLCollection (1)
// ...il documento viene esteso con altri <a>...
console.log(links); // HTMLCollection (3): la collezione si è aggiornata
    da sola
```

Invocati su `document`, i metodi di ricerca cercano nell'**intero documento**; se gli elementi cercati sono discendenti di un altro elemento, è più efficiente invocarli **sull'elemento antenato**.

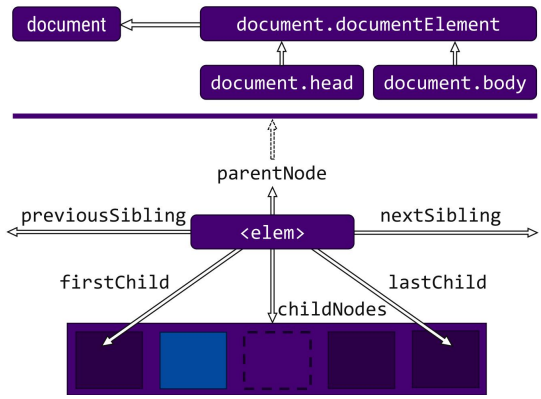
```JavaScript
let sidebar = document.querySelector(".sidebar");
let links = sidebar.querySelectorAll("a"); // cerca solo dentro sidebar
```

| Metodo | Cerca per… | Chiamabile sugli elementi? | Restituisce live collection? |
|---|---|---|---|
| `querySelector` | selettore CSS | ✓ | – |
| `querySelectorAll` | selettore CSS | ✓ | – |
| `getElementById` | attributo `id` | – | – |
| `getElementsByName` | attributo `name` | – | ✓ |
| `getElementsByTagName` | nome del tag opp * | ✓ | ✓ |
| `getElementsByClassName` | attributo `class` | ✓ | ✓ |

### Navigare nel DOM

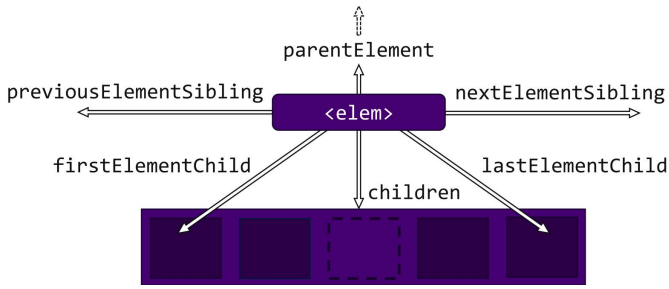
Gli elementi più in alto (`html`, `head`, `body`) sono disponibili anche come **proprietà di `document`**; inoltre, i nodi del DOM sono collegati tra loro da riferimenti di navigazione.

- I nodi del DOM contengono **riferimenti a padre, fratelli e figli**: `parentNode`, `previousSibling` / `nextSibling`, `firstChild` / `lastChild`, `childNodes`.
- Questi riferimenti sono **di sola lettura**.




### Navigazione solo elementi

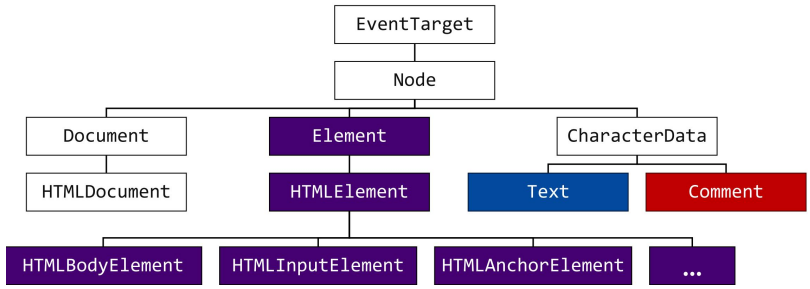
Per ignorare i nodi di testo e commento si usano le proprietà che considerano **solo i DOM element**: `parentElement`, `children`, `firstElementChild` / `lastElementChild`, `previousElementSibling` / `nextElementSibling`.



```HTML
<main><h1>Hello DOM!</h1><!-- comment --><p>DOM is <em>FUN</em>!</p></main>
<script>
  document.querySelector("main").childNodes; // [#text, h1, #text, #comment, #text, p, #text]
  document.querySelector("main").children;   // HTMLCollection { h1, p }
</script>
```

### Gerarchia delle classi dei nodi

I nodi del DOM appartengono a una **gerarchia di classi**: da `EventTarget` deriva `Node`, da cui derivano `Document`, `Element` e `CharacterData` (con `Text` e `Comment`); da `HTMLElement` derivano le classi specifiche di ogni tag (`HTMLInputElement`, `HTMLAnchorElement`, …).




### Proprietà dei nodi

#### `nodeName` / `tagName`

I nodi del DOM hanno alcune proprietà utili; le prime riguardano il tipo di nodo:

- `nodeName` e `tagName` danno informazioni sul **tipo di nodo** e sono di **sola lettura**.
- Per gli elementi valgono il nome del tag (es. `"H1"`); per i nodi di testo e commento valgono `#text` e `#comment`, mentre `tagName` è `undefined` (non sono elementi).

```JavaScript
let body = document.body;
body.firstChild.nodeName;          // "#text"
body.firstChild.tagName;           // undefined (non è un elemento)
body.childNodes[3].nodeName;       // "#comment"
body.firstElementChild.tagName;    // "H1"
```

### Attributi come proprietà

Gli attributi **standard** vengono analizzati e resi disponibili come **proprietà** dell'elemento corrispondente; gli attributi non standard non lo sono (per quelli si usa `getAttribute`).

```JavaScript
// <input id="x" class="inp ok" name="field" custom="y">
let i = document.body.firstElementChild;
i.id;                      // "x" — attributo standard
i.classList;                // DOMTokenList ["inp", "ok"]
i.custom;                   // undefined — non è un attributo standard
i.getAttribute("custom");  // "y"
```

Queste proprietà sono anche **scrivibili**:

```JavaScript
i.id = "w";
i.classList.remove("ok"); i.classList.add("err");
i.setAttribute("custom", "k");
i.removeAttribute("name");
// → <input id="w" class="inp err" custom="k">
```

### `innerHTML` / `outerHTML`

Un'altra coppia di proprietà riguarda il codice HTML dell'elemento:

- `innerHTML` restituisce il codice HTML **contenuto** in un elemento; `outerHTML` il codice HTML dell'**intero elemento**.
- Sono entrambi **scrivibili**: assegnare loro un valore cambia la pagina — `innerHTML` cambia il contenuto, `outerHTML` sostituisce l'elemento.

```JavaScript
let h1 = document.querySelector("h1"); // <h1 id="h">Hello <em>DOM!</em></h1>
h1.innerHTML; // "Hello <em>DOM!</em>"
h1.outerHTML; // "<h1 id=\"h\">Hello <em>DOM!</em></h1>"

h1.innerHTML = "DOM: <em>Wow!</em>";      // nuovo contenuto dell'elemento
h1.outerHTML = `<p>${h1.innerHTML}</p>`;  // l'<h1> diventa un <p>, stesso contenuto
```

## Manipolare il DOM

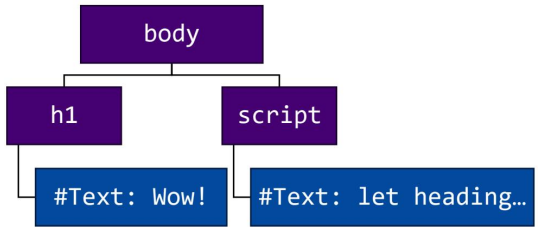
Non ci si limita a modificare gli elementi esistenti (es. cambiandone gli attributi): la struttura stessa del DOM può cambiare.

- Si possono **creare** nuovi elementi e **rimuovere** quelli esistenti, cambiando la **struttura del DOM**.
- L'aggiornamento dinamico del DOM è la chiave per pagine web reattive, «vive».

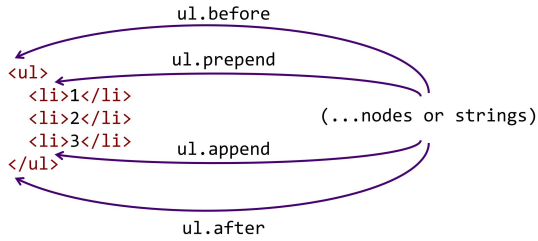
### Creare nuovi elementi

`document.createElement(tag)` crea un nuovo elemento, mentre `document.createTextNode(value)` crea un nuovo nodo `#text`; i nodi appena creati **non sono ancora nel DOM** e vanno inseriti con i metodi di inserimento.

```JavaScript
let heading = document.createElement("h1");  // nuovo <h1>, non ancora nel DOM
let title = document.createTextNode("Wow!"); // nuovo #text, non ancora nel DOM
heading.append(title);          // #text come figlio dell'<h1>
document.body.prepend(heading); // <h1> (con il suo figlio) in cima al <body>
```




### Metodi di inserimento




| Metodo | Descrizione |
|---|---|
| `document.createElement(tag)` | crea un elemento con il tag indicato |
| `document.createTextNode(value)` | crea un nodo di testo (raramente usato) |
| `elem.cloneNode(deep)` | clona l'elemento; se `deep == true` con tutti i discendenti |
| `node.append(...nodi o stringhe)` | inserisce dentro il nodo, alla fine |
| `node.prepend(...nodi o stringhe)` | inserisce dentro il nodo, all'inizio |
| `node.before(...nodi o stringhe)` | inserisce subito prima del nodo |
| `node.after(...nodi o stringhe)` | inserisce subito dopo il nodo |
| `node.replaceWith(...nodi o stringhe)` | sostituisce il nodo |
| `node.remove()` | rimuove il nodo |

### Interagire con l'utente

**`alert`**, **`prompt`** e **`confirm`** sono funzioni del browser utili per interagire con l'utente.

- `alert(message)` mostra un messaggio all'utente.
- `confirm(question)` chiede una conferma e restituisce `true`/`false`.
- `prompt(question)` chiede un input e restituisce la stringa inserita dall'utente.

```JavaScript
alert("This page will ask for your input");
if (confirm("Do you want to continue?") === true) {
  let name = prompt("State your name:");
} else {
  console.log("user cancelled");
}
```

# Eventi

### Definizione e classificazione

Definition

**Evento**

Un **evento** è un segnale che indica che è successo qualcosa.

Gli eventi possono essere legati a diverse aree:

- **Comportamento dell'utente**: click, pressioni di tasti sulla tastiera, movimenti del mouse.
- **Form**: invio (submission), focus (o perdita di focus) su un campo di input.
- **Eventi del documento**: documento o elementi caricati, richieste di rete completate.

| Categoria | Evento | Significato |
|---|---|---|
| Mouse | `click` | l'utente clicca (tap su dispositivi touchscreen) su un elemento |
| | `contextMenu` | clic destro del mouse su un elemento |
| | `mouseover` / `mouseout` | il cursore entra in / esce da un elemento |
| | `mousedown` / `mouseup` | il pulsante principale del mouse è premuto / rilasciato su un elemento |
| | `mousemove` | il cursore del mouse si muove |
| Keyboard | `keydown` / `keyup` | un tasto della tastiera è premuto / rilasciato |
| Forms | `submit` | l'utente invia un form |
| | `focus` | l'utente dà focus a un elemento (es. `<input>`) |
| Document | `DOMContentLoaded` | l'HTML è stato analizzato, il DOM è completamente costruito |

| Categoria | Evento | Significato |
|---|---|---|
| Window | `load` | la pagina è stata completamente caricata e renderizzata |

### Handler

Per reagire agli eventi si definiscono **handler**: funzioni eseguite quando un evento scatta.

- Il modo più semplice per definirli sono gli **attributi HTML** `on<evento>`, il cui valore è il codice JavaScript da eseguire.

```HTML
<input type="button" value="Click me" onclick="handleClick();">
```

Gli handler si possono anche definire **dinamicamente** via JavaScript, in due modi:

- Assegnando la **proprietà** `on<evento>` dell'elemento: **sovrascrive** l'handler definito tramite attributo HTML.
- Con il metodo `addEventListener`: invece **aggiunge** un nuovo handler.

```JavaScript
input.onclick = () => console.log("Tap");     // sovrascrive l'handler da attributo
input.addEventListener("click", handleClick); // aggiunge una nuova funzione handler
```

### `this` negli handler

Dentro un event handler, `this` vale l'elemento a cui l'handler invocato è **assegnato**.

```JavaScript
function handleClick(){ console.log(this.outerHTML); }
// assegnato all'<input>: stampa l'<input>; assegnato al <p>: stampa il <p>
```

### L'oggetto event

Tutti i dettagli dell'evento (es.: quale tasto è stato premuto? a quali coordinate è avvenuto il click?) sono forniti da un **oggetto event**, passato agli handler. Ogni tipo di evento ha le sue **proprietà**: es. gli eventi legati alla tastiera hanno la proprietà `key` con il tasto premuto.

```JavaScript
function handler(event){ console.log(`You pressed ${event.key}`); }
document.querySelector("input").addEventListener("keyup", handler);
```

### Event bubbling

Definition

**Event bubbling**

Quando un evento scatta su un elemento, prima vengono eseguiti tutti gli handler registrati **su quell'elemento**, poi quelli del **padre**, e così via risalendo tutti gli antenati fino al **nodo radice** del DOM.

Il meccanismo ha due caratteristiche da tenere a mente:

- Si applica alla **maggior parte** degli eventi, ma non a tutti: gli eventi `focus` non fanno bubbling.
- Un evento in bubbling si propaga fino all'oggetto globale `window`.

HTML

```
<section onclick="console.log('section');">
  <article onclick="console.log('article');">
    <p onclick="console.log('p');">Bubbling</p>
  </article>
</section>
<!-- output in console dopo un click su <p>: p, article, section,
     document -->
```

### Event delegation

Info

**Event delegation**

Se molti elementi possono generare eventi simili, invece di assegnare un handler a ciascuno se ne usa **uno solo sul loro antenato comune**: gli elementi **delegano** la gestione dell'evento a un antenato.

- La proprietà `target` dell'oggetto event permette di determinare **su quale elemento** l'evento è stato generato.

HTML

```
<table onclick="handler(event);"><!-- ...righe con celle <td>... --></
    table>
<script>
  function handler(event){
    if (event.target.nodeName === "TD") {  // gestisce solo le celle
      let oldValue = parseInt(event.target.innerHTML);
      event.target.innerHTML = ++oldValue;
    }
  }
</script>
```

### Fermare il bubbling

- Un handler può decidere che l'evento è stato **completamente processato** e non va propagato oltre: il bubbling si ferma invocando `stopPropagation()` sull'oggetto event.

Warning

**Regola pratica**

Non fermare la propagazione senza un buon motivo: in futuro potrebbe servire catturare l'evento su un antenato.

```JavaScript
function handler(event){
  console.log(this.tagName);
  event.stopPropagation();
}
```

### Generare eventi (dispatching)

- Da JavaScript è possibile **generare eventi**: con il costruttore `new Event(...)` e il metodo **`dispatchEvent`**.

Info

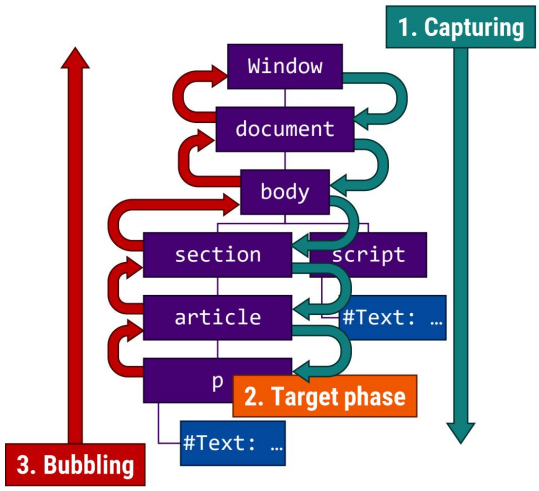
**isTrusted**

La proprietà **`isTrusted`** dell'evento vale **`false`** per gli eventi «artificiali» generati via JavaScript.

```JavaScript
let click = new Event("click");
input.dispatchEvent(click); // isTrusted === false per gli eventi
    generati via JS
```

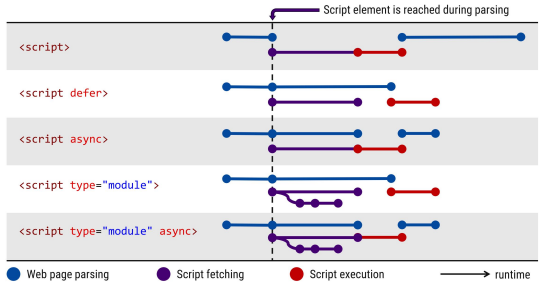
### Capturing

- Esiste un'altra fase di elaborazione dell'evento, il **capturing**, che procede «verso il basso» fino all'elemento target.
- L'elaborazione di un evento ha tecnicamente **tre fasi**: 1. capturing, 2. target, 3. bubbling.
- I listener nella fase di capturing si registrano con `elem.addEventListener(..., { capture : true });`.




## Ordine di esecuzione degli script

- Man mano che gli script diventano complessi, interagiscono con il DOM e generano dinamicamente handler, serve conoscere **come viene eseguito il codice JavaScript** al caricamento di una pagina.
- **Default**: gli script vengono scaricati ed eseguiti **nell'ordine in cui compaiono**, mentre il parsing della pagina è **in pausa**.
- `defer`: gli script esterni vengono scaricati **in parallelo** ed eseguiti **dopo** che la pagina ha finito di essere analizzata.
- `async`: gli script esterni vengono scaricati in parallelo ed eseguiti **appena scaricati**.
- Gli script **module** sono `defer` di default e possono essere dichiarati `async`.




- L'ordine di esecuzione **conta**: uno script nel `<head>` senza attributi viene eseguito prima che il `<body>` esista, quindi `document.body` è `null` (TypeError).

| Script nel `<head>` che accede a un elemento del `<body>` | Esito |
|---|---|
| `<script src="load.js">` | eseguito subito: `document.body` è `null` → TypeError |
| `<script defer src="load.js">` | eseguito a parsing concluso: il DOM è pronto ✓ |

| Script nel `<head>` che accede a un elemento del `<body>` | Esito |
|---|---|
| `<script async src="load.js">` | non prevedibile: dipende dal tempo di scaricamento |
| handler di `DOMContentLoaded` | eseguito quando il DOM è completamente costruito ✓ |

JavaScript

```
document.addEventListener("DOMContentLoaded", () => {
  let h1 = document.body.querySelector("h1"); // ora il body esiste
  console.log(h1.innerHTML);
});
```

Capitolo 11

# Lezione 11 JavaScript asincronismo e richieste di rete

## Storage dei dati nel browser

I browser moderni forniscono diverse API, accessibili via JavaScript, per memorizzare e recuperare dati dell'applicazione: **cookie**, **Web Storage** (`localStorage` e `sessionStorage`) e **IndexedDB**.

### Cookie

Definition

**Cookie**

**Piccole stringhe di dati** appartenenti al protocollo HTTP, usate per «superarne» l'assenza di stato (*statelessness*). Vengono tipicamente impostati in una risposta HTTP tramite l'intestazione `Set-Cookie`; il browser li memorizza e li invia in tutte le richieste successive allo stesso dominio, tramite l'intestazione `Cookie` della richiesta.

I cookie del sito corrente sono accessibili da JavaScript tramite la proprietà `document.cookie`.

- Il valore di `document.cookie` è una stringa di coppie `name=value` delimitate da `;`.
- Ogni coppia è un cookie separato.

JavaScript

```
1  document.cookie = `name=${name}; max-age=10`; // scade dopo 10 secondi
```

### localStorage

Gestire i cookie via JavaScript non è molto immediato, soprattutto con più cookie: occorre manipolare stringhe, separarle, usare espressioni regolari. L'oggetto `localStorage` fornisce modi semplici per memorizzare coppie chiave/valore nel browser.

- I dati non vengono inviati con ogni richiesta, quindi si possono memorizzare molti più dati.
- È abbastanza simile a una `Map` e può memorizzare solo valori stringa.
- Esiste un oggetto `localStorage` diverso per ogni combinazione di dominio/protocollo/porta.

### sessionStorage

L'oggetto `sessionStorage` fornisce le stesse funzionalità di `localStorage`, ma è usato molto più raramente.

- A differenza di `localStorage`, i cui dati sopravvivono anche a un riavvio completo del browser, `sessionStorage` esiste solo nella scheda corrente del browser.

- I dati sopravvivono al ricaricamento della pagina, ma non alla chiusura e riapertura della scheda.

### IndexedDB

Definition

**IndexedDB**

**Database integrato nel browser**, che supporta diversi tipi di chiave, transazioni e query per intervalli di chiavi (*key-range queries*), e può memorizzare ancora più dati di `localStorage`.

Eccessivamente potente per le tradizionali applicazioni client-server: è pensata soprattutto per le applicazioni offline.

## Sincronismo e asincronismo

### Codice sincrono

Tipicamente il codice JavaScript viene eseguito in modo **sincrono**: ogni istruzione attende il completamento delle precedenti.

- È un modo intuitivo di programmare, ma ha alcuni svantaggi.
- Alcune istruzioni possono richiedere tempo per completarsi: attesa di input dall'utente, attesa del completamento di una richiesta di rete, attesa del termine di un calcolo complesso.
- Le istruzioni successive, anche se importanti, possono rimanere bloccate in attesa: l'**asincronismo** aiuta ad affrontare questi problemi.

### Codice asincrono

L'esecuzione di JavaScript è ordinariamente **single-threaded**: è il browser a fornire le capacità asincrone, come i timer e le richieste di rete.

JavaScript esegue le callback quando può, una volta completate le operazioni pertinenti.

JavaScript

```
1 console.log("A");
2 setTimeout(() => { console.log("B"); }, 2000); // dopo 2000 ms
3 console.log("C"); // l'output è "A", "C" e infine "B"
```

### Callback

Definition

**Callback**

**Funzioni passate come argomenti** ad altre funzioni, con l'aspettativa che vengano invocate al momento appropriato. I gestori di eventi (*event handler*) sono infatti callback.

### Limiti delle callback

Quando si gestiscono molte callback annidate dentro altre callback, il codice diventa molto disordinato molto rapidamente, e la gestione degli errori lo complica ulteriormente.

JavaScript

```
firstOperation(result, (r1) => {
  secondOperation(r1, (r2) => {
    thirdOperation(r2, (r3) => { console.log(r3); });
  });
});
```

- Per questo si parla di **Callback Hell** o *Pyramid of Doom*.
- Il vecchio JavaScript gestiva l'asincronismo con le callback; in quello moderno si usa tipicamente un nuovo costrutto: la **Promise**.

# Promise

### Produttore e consumatore

In programmazione capita spesso che esistano un codice **produttore**, che fa qualcosa che richiede tempo (attende input dell'utente, scarica dati dalla rete, esegue calcoli complessi), e un codice **consumatore**, che ha bisogno dei risultati del produttore.

Definition

**Promise**

**Costrutto del JavaScript moderno** che serve a collegare produttori e consumatori.




### Creazione di una Promise

Un oggetto `Promise` si crea con l'apposito costruttore, passando come argomento l'**executor**: la funzione viene invocata immediatamente, appena la Promise viene creata.

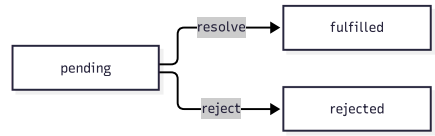
- `resolve` e `reject` sono callback fornite da JavaScript.
- Quando il codice dell'executor termina, deve chiamare una delle due.
- `resolve(value)`: se l'operazione è completata con successo, con risultato `value`.
- `reject(error)`: se si è verificato un errore, con l'oggetto errore `error`.

JavaScript

```
let promise = new Promise(function(resolve, reject) {
  // executor: qui il codice produttore
});
```

### Ciclo di vita

Una Promise nasce nello stato **pending**; quando viene stabilita (*settled*) diventa **fulfilled** (mantenuta) oppure **rejected** (rifiutata).




```JavaScript
console.log(promise); // Promise { <state>: "pending" }
```

### then

Il codice consumatore riceve una Promise che certi dati saranno resi disponibili in un certo momento del futuro, e registra le azioni da eseguire tramite metodi specifici offerti dall'oggetto Promise; il più importante è `.then()`.

- `.then()` prende in input due callback: la prima è invocata quando la Promise è mantenuta, la seconda quando è rifiutata.
- È anche possibile registrare solo la funzione da eseguire in caso di Promise mantenuta.

```JavaScript
promise.then(
  (result) => { /* Promise mantenuta */ },
  (error) => { /* Promise rifiutata */ }
);
```

### catch

Quando interessano solo gli scenari di errore, si può passare `null` come primo argomento di `.then()`, oppure usare il metodo equivalente `.catch()`.

```JavaScript
promise.catch((error) => { /* Promise rifiutata */ });
```

### finally

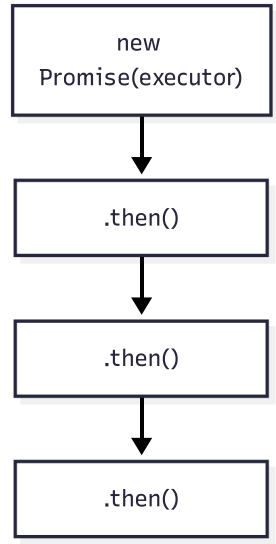
Come i blocchi `try/catch`, anche le Promise offrono un metodo `.finally()`, che prende una callback senza input.

- È invocato appena la Promise si stabilisce, sia con `resolve` sia con `reject`.
- È pensato per eseguire pulizie (*clean-up*) e altre operazioni da svolgere a prescindere dallo stato della Promise.

### Chaining

Quando serve eseguire una sequenza di passaggi asincroni si usa il **concatenamento** (*chaining*) delle Promise: la Promise originale viene risolta una sola volta, ma ogni invocazione di `.then()` restituisce in realtà una nuova Promise, che rappresenta il completamento del gestore stesso.

- Quando un gestore restituisce un valore, questo diventa il risultato della nuova Promise.
- Un `throw` all'interno di un gestore equivale a una `reject()` implicita.




JavaScript

```
promise
  .then((r) => { console.log(r); return r * 2; }) // 1
  .then((r) => { console.log(r); return r * 2; }) // 2
  .then((r) => { console.log(r); return r * 2; }); // 4
```

Error

**Concatenamento errato**

Chiamare più volte `.then()` sulla stessa Promise non è concatenamento: ogni gestore riceve il risultato della Promise originale, non il valore restituito dal gestore precedente.

JavaScript

```
// con risultato originale 1: stampa 1, 1, 1
promise.then((r) => { console.log(r); return r * 2; });
promise.then((r) => { console.log(r); return r * 2; });
```

### Metodi statici dell'API Promise

L'oggetto `Promise` espone anche metodi statici per lavorare con più Promise contemporaneamente.

- **`Promise.all(array)`**: restituisce una nuova Promise che rappresenta il completamento di tutte le Promise di ingresso; il risultato è un array in cui ogni Promise di ingresso contribuisce un elemento; se una sola Promise viene rifiutata, `Promise.all` si rifiuta immediatamente.
- **`Promise.allSettled(array)`**: aspetta che tutte le Promise di ingresso si stabilizzano, ma restituisce come risultato un array di oggetti che rappresentano lo stato di ciascuna (`status: "fulfilled"` con `value`, oppure `status: "rejected"` con `reason`).

- **`Promise.race(array)`**: simile a `Promise.all()`, ma si stabilisce appena una delle Promise di ingresso si stabilizza, e ne assume il risultato o l'errore.
- **`Promise.any(array)`**: simile a `Promise.race()`, ma attende la prima Promise risolta; se tutte le Promise di ingresso vengono rifiutate, si rifiuta con un `AggregateError` contenente tutti gli errori.
- **`Promise.resolve(value)`** e **`Promise.reject(error)`**: creano Promise già stabilite (rispettivamente risolta o rifiutata); a volte sono usate per compatibilità, ad esempio quando una funzione è attesa che restituisca una Promise.

JavaScript

```
Promise.all([
  new Promise(res => setTimeout(() => res(3), 3000)),
  new Promise(res => setTimeout(() => res(2), 2000)),
  new Promise(res => setTimeout(() => res(1), 1000)),
]).then((result) => console.log(result)); // [3, 2, 1]
```

## async e await

Il JavaScript moderno include una sintassi speciale per lavorare con le Promise, composta da due parole chiave: `async` e `await`.

### async

La parola chiave `async`, posta prima di una funzione, garantisce che la funzione restituisca sempre una Promise.

- Qualsiasi altro valore restituito da una funzione `async` viene implicitamente incapsulato in una Promise risolta.
- Gli errori lanciati vengono incapsulati in una Promise rifiutata.

JavaScript

```
async function g() {
  return 1; // equivale a restituire Promise.resolve(1)
}
```

### await

La parola chiave `await` si può usare solo all'interno di funzioni `async`, e mette in pausa l'esecuzione del codice finché una Promise non si stabilisce.

JavaScript

```
async function f() {
  let promise = new Promise((res) => setTimeout(() => res("Done"), 2000))
    ;
  let result = await promise; // l'esecuzione si mette in pausa qui
  return result;
```

```
}
```

# Richieste di rete

JavaScript può anche scaricare dati da Internet.

### XMLHttpRequest

Nel vecchio JavaScript i programmatori usavano oggetti `XMLHttpRequest`, con callback per gestire gli eventi di stato.

### fetch

**Definition**

**fetch()**

**Modo moderno di eseguire richieste HTTP** in JavaScript: prende in ingresso un URL e un array opzionale di opzioni, e restituisce una Promise che si risolve in un oggetto `Response`.

JavaScript

```
let promise = fetch("url", [options]);
```

Le opzioni permettono di specificare metodo HTTP, intestazioni della richiesta, eccetera.

### Lavorare con la risposta

**Warning**

**Risoluzione anticipata**

La Promise restituita da `fetch()` si risolve appena arrivano le intestazioni della risposta: in quel momento si possono controllare stato e intestazioni HTTP, ma il corpo della risposta potrebbe non essere ancora disponibile.

Sull'oggetto `Response` si possono controllare stato e intestazioni.

- `response.status` contiene il codice di stato HTTP.
- `response.ok` indica se la richiesta ha avuto successo.
- `response.headers.get('Content-Type')` legge una specifica intestazione; le intestazioni della risposta sono iterabili.

JavaScript

```
let response = await fetch("./example.json");
console.log(response.headers.get('Content-Type'));
if (response.ok) { /* richiesta riuscita */ } else { /* errore */ }
```

### Corpo della risposta

L'oggetto `Response` fornisce diversi metodi basati su Promise per accedere al corpo in vari formati, non appena diventa disponibile: ad esempio `.json()` interpreta il corpo come JSON.

| Metodo | Risultato |
|---|---|
| `.text()` | Legge il corpo della risposta e lo restituisce come testo |
| `.json()` | Interpreta il corpo della risposta come JSON |
| `.blob()` | Restituisce un Blob (dati binari con tipo) |
| `.arrayBuffer()` | Restituisce un ArrayBuffer (rappresentazione binaria di basso livello) |
| `formData()` | Restituisce un FormData (dati inviati tramite form) |

JavaScript

```
let response = await fetch("https://catfact.ninja/fact");
if (response.ok) {
  let fact = await response.json(); // corpo interpretato come JSON
}
```

Warning

**Corpo consumabile una sola volta**

Si può scegliere **un solo** metodo di lettura del corpo: se è già stato chiamato `response.text()`, `response.json()` non funziona perché il contenuto del corpo è già stato consumato.

# Parte VI

# Server Side

Capitolo 12

# Lezione 12 Web statico e dinamico

## Richiami su Promise e Fetch

### Promise

Un rapido richiamo sul ciclo di vita e sul consumo delle promise.

- **Creazione**: `new Promise(executor)` crea la promise; l'executor riceve le funzioni `resolve` e `reject` per stabilirne l'esito.
- **`setTimeout`**: prende in input una callback e un tempo in millisecondi.
- **Consumo**: `.then(onResult, onError)` gestisce rispettivamente successo ed errore; `.finally()` esegue una callback quando la promise si è *settled*.
- **Asincronia**: le callback della promise vengono eseguite dopo il codice sincrono: `Here` e `There` vengono stampati prima di `Promise settled` e del risultato.

```JavaScript
let promise = new Promise((resolve, reject) => {
    // l'executor chiama resolve(value) o reject(new Error(...))
});
promise.finally(() => { /* eseguita quando la promise si settle */ });
promise.then(
    (result) => { /* successo */ },
    (error) => { /* errore: error.message */ });
```

### Fetch API

`fetch()` restituisce una promise che risolve con un oggetto `Response`.

- **Header specifico**: si legge con `response.headers.get(name)`.
- **Tutti gli header**: `response.headers` è iterabile con `for(let [key, value] of response.headers)`.
- **Stato**: `response.status` contiene il codice di stato HTTP; `response.ok` è `true` quando la richiesta ha avuto successo.
- **Corpo**: quando la promise di `fetch()` si risolve, il corpo potrebbe non essere ancora disponibile; la `Response` fornisce metodi *promise-based* per accedere al corpo in vari formati.
  - **`.json()`**: elabora il corpo come JSON appena diventa disponibile.

```JavaScript
async function f(){
    let response = await fetch("./example.json");
```

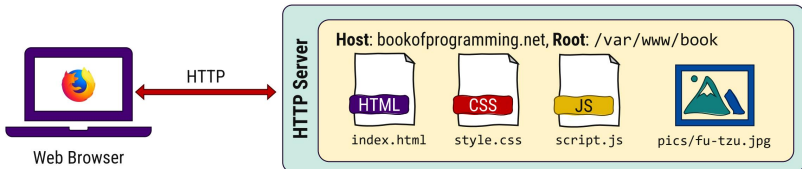
```
    response.headers.get('Content-Type');   // un singolo header
    for(let [key, value] of response.headers){ /* iterazione */ }
    let status = response.status;           // codice di stato
    if(response.ok){ /* richiesta riuscita */ }
    let fact = await response.json();       // corpo come JSON
}
```

# Il web statico

Finora abbiamo imparato le tecnologie core del web — **HTML**, **CSS** e **JavaScript** — progettando pagine moderne, responsive e dinamiche: gestione di eventi, modifica del DOM al volo e richieste HTTP asincrone per recuperare dati da mostrare nelle pagine.

### App web statiche

- **Dinamicità nel browser**: il dinamismo dato da JavaScript avviene interamente dentro il browser.
- **Documento identico**: ogni richiesta a una stessa pagina riceve sempre lo stesso documento identico, servito dal **document root** del server.




### Limitazioni del web statico

L'approccio statico è semplice ed efficace per applicazioni con un numero limitato di pagine che cambiano raramente, ma soffre di alcuni limiti.

- **Personalizzazione**: non permette di adattare una pagina a un utente specifico.
- **Contenuti variabili**: non è adatto quando il contenuto di una pagina cambia molto frequentemente.
- **Scala**: non regge siti costituiti da milioni di pagine.
- **Azioni**: non permette di agire in base a una richiesta (es. salvare un ordine).

Info

**Costo della duplicazione**

In un sito di ~60 milioni di pagine con la stessa struttura da ~150kB ripetuta in ogni pagina si accumulano ~9TB di dati duplicati; un elemento comune (es. la *navbar*) andrebbe modificato in tutte le pagine.

# Programmazione server-side

### Generazione on-the-fly

Un web server può fare molto più che servire file: può **generare** pagine web **on-the-fly**, tipicamente in base ai parametri ricevuti nella richiesta HTTP. È l'idea chiave dietro la **programmazione server-side**.

- **Parametri della richiesta**: richieste diverse, anche verso la stessa risorsa, producono pagine diverse.
- **Contenuti sempre aggiornati**: la pagina viene costruita al momento della richiesta, non preparata in anticipo.




# CGI

### Definizione

Definition

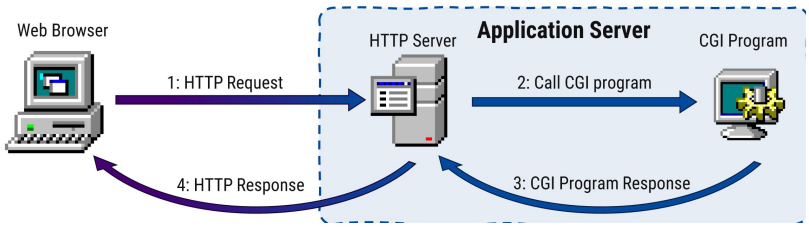
**Common Gateway Interface**

**Specifica di interfaccia** che consente a un web server di eseguire programmi esterni per processare una richiesta HTTP. Diffusa nei primi anni '90, prima ancora dell'esistenza di JavaScript, fu il primo strumento per ottenere pagine web dinamiche.

### Flusso di una richiesta

Il CGI definisce come comunicano il web server e i programmi **CGI-compliant**.

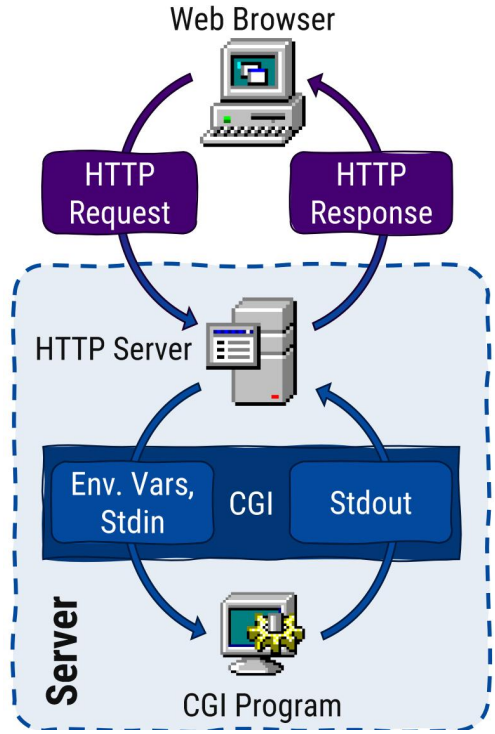
- **Browser → Server**: il browser invia la richiesta HTTP al web server.
- **Server → Programma**: il server invoca il programma CGI per processarla.
- **Ritorno**: l'output del programma torna al server, che lo invia al browser come risposta HTTP.



### Input e output del programma

Il programma CGI accede ai suoi input e costruisce la risposta secondo regole precise.

- **Header e query string della richiesta**: disponibili tramite **variabili d'ambiente**.
- **Corpo della richiesta**: disponibile al programma sullo stream **stdin**.
- **Corpo della risposta**: è l'output del programma sullo stream **stdout**.




### Primo script CGI

Il primo script CGI mostra la struttura essenziale della risposta: prima gli header (es. `Content-type`), poi una riga vuota, poi il corpo HTML; un contenuto generato al volo (la data corrente) dimostra la dinamicità della pagina.

```bash
#!/bin/bash
echo "Content-type: text/html; charset=utf-8"
echo ""
echo "<h1>Hello CGI!</h1>"
date +"%d/%m/%Y at %H:%M:%S"
```

## Server-side scripting

### Limiti dell'approccio CGI

Scrivere programmi CGI non è un'esperienza piacevole.

- **HTML su stdout**: scrivere interi documenti HTML sullo standard output è macchinoso.

- **Parsing manuale**: analizzare a mano query string e corpi delle richieste è tutt'altro che divertente.
- **Processi**: creare un nuovo processo per ogni richiesta non è molto efficace in termini di risorse.

### Linguaggi di scripting server-side

Per fortuna sono emersi linguaggi e framework specializzati per lo scripting server-side.

- **PHP**: acronimo ricorsivo di *PHP Hypertext Preprocessor*, 1995.
- **ASP**: *Active Server Pages*, linguaggio di scripting server-side di Microsoft, 1997.
- **JSP**: *Java Server Pages*, rilasciato originariamente da Sun, 1999.

L'idea era «mixare» codice server-side e HTML: un interprete speciale elabora il mix per produrre codice HTML «puro».

## PHP

### Delimitatori

PHP usa **delimitatori** speciali per separare codice e HTML.

- `<?php ... ?>`: racchiudono il codice PHP.
- **Testo esterno ai delimitatori**: resta così com'è nell'output.
- **Codice interno ai delimitatori**: viene interpretato e l'output risultante è inserito nell'HTML.
- `<?=`: shorthand per `<?php echo`.

PHP

```php
<title><?php echo "Hello PHP" ?></title>
<?php $array = ["Hello", "PHP"] ?>
<ul>
    <?php foreach($array as $item): ?>
        <li><?= $item ?></li>
    <?php endforeach; ?>
</ul>
```

### Gestione delle richieste

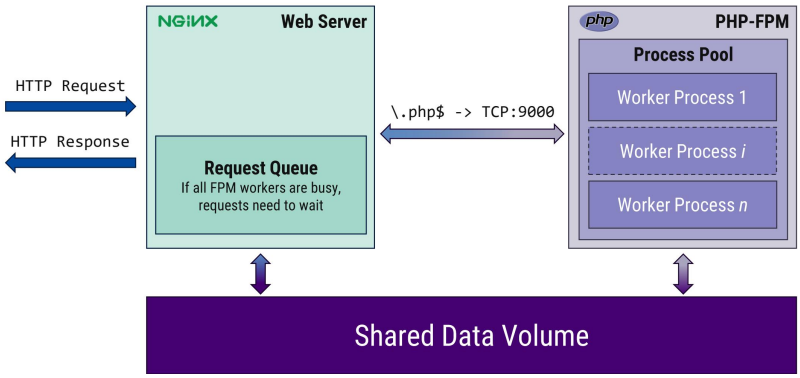
Le soluzioni di scripting server-side supportano lo sviluppatore nei compiti tediosi tipici della gestione delle richieste HTTP: le richieste vengono pre-processate automaticamente.

- **Parametri della richiesta**: disponibili negli array associativi `$_GET` e `$_POST`; ad esempio `$_POST['todo']` accede al parametro `todo`.
- **Cookie**: già elaborati nell'array associativo `$_COOKIES`.
- **Risposta**: non è necessario scrivere esplicitamente l'intera risposta su `stdout`.

### Architettura NGINX e PHP-FPM

Un'applicazione PHP moderna può essere distribuita con **NGINX** come web server e **PHP-FPM** (*FastCGI Process Manager*) come gestore dei processi PHP.

- **NGINX**: web server che riceve le richieste HTTP in arrivo e le smista.
- **Pool di worker**: PHP-FPM gestisce un pool di processi worker che eseguono il codice PHP.
- **Comunicazione**: NGINX inoltra a PHP-FPM le richieste PHP via TCP sulla porta `9000`; i due componenti condividono un volume di dati comune.


NGINX
Web Server
HTTP Request
HTTP Response
Request Queue
If all FPM workers are busy, requests need to wait
\.php$ -> TCP:9000
php
PHP-FPM
Process Pool
Worker Process 1
Worker Process i
Worker Process n
Shared Data Volume

Capitolo 13

# Lezione 13 Node.js

## JavaScript in ambiente server

Col *server-side scripting* le pagine web possono essere generate al volo da programmi in esecuzione sul server; JavaScript, finora usato solo in un ambiente browser, può essere eseguito anche in un ambiente server.

Per eseguire JavaScript lato server sono disponibili diversi **runtime**, tra cui **Node.js**, **Deno** e **Bun**.

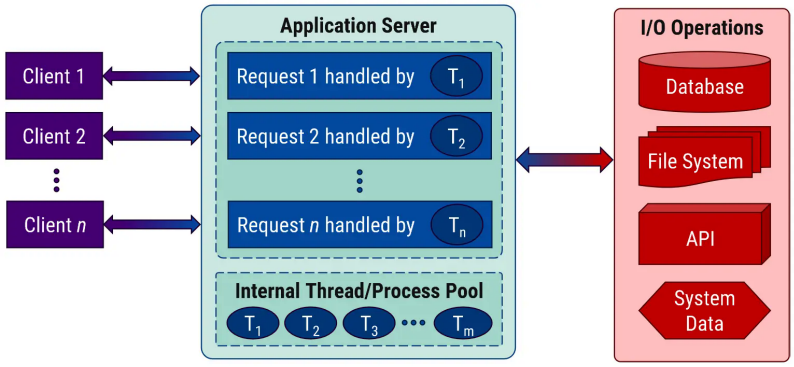
Definition

**Node.js**

**Runtime Environment** per JavaScript *open-source* e *cross-platform*, che esegue il motore **V8** di Google proveniente dal progetto Chromium. Si basa su un modello *event-based*, asincrono per default e a I/O *non-blocking*, ed è di gran lunga l'ambiente di esecuzione JavaScript più popolare.

## Gestione multi-threaded delle richieste

Negli altri ambienti di esecuzione e linguaggi, ogni richiesta è gestita da un thread o processo dedicato, prelevato da un pool interno all'*application server*.




Nel servire le richieste, le web application devono spesso eseguire operazioni I/O **bloccanti** e time-consuming: **accedere a un database**, **accedere a un'API esterna** (es. con `fetch()`), **leggere un file** dal file system locale, **leggere il body della richiesta**. Di conseguenza, i thread o processi che gestiscono una richiesta passano molto tempo in attesa del completamento di queste operazioni.

Il modello multi-threaded ha due limiti:

- **Inefficienza**: gran parte del tempo dei thread è speso in attesa delle operazioni bloccanti.

- **Scalabilità limitata**: il thread pool è pre-allocato a una dimensione predefinita, quindi non si possono gestire più richieste concorrenti di quante il pool ne permetta.

# La filosofia di Node.js

Node.js adotta una filosofia alternativa al modello multi-threaded, orientata all'efficienza.

- Un programma Node.js viene eseguito in **un singolo processo**: non serve creare un nuovo thread per ogni richiesta.
- Il runtime fornisce nella standard library un insieme di **primitive di I/O asincrone** che impediscono al codice JavaScript di bloccarsi.
- La maggior parte delle librerie sfrutta il paradigma *non-blocking* ogni volta che è possibile.
- Vasta adozione del **callback pattern**: la funzione da eseguire al termine dell'operazione viene registrata al momento della chiamata.
- Le operazioni sincrone sono generalmente un'eccezione più che la norma.

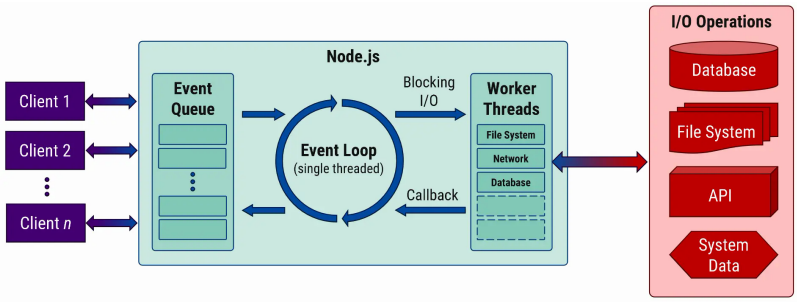
### Il single-threaded event loop

**Definition**

**Event loop**

**Ciclo a thread singolo** al centro del runtime Node.js: preleva gli eventi in coda, esegue i callback e delega le operazioni I/O bloccanti ai *worker thread*.

- **Event queue**: le richieste dei client arrivano come eventi in coda, in attesa di essere processati.
- **Worker thread**: eseguono le operazioni I/O bloccanti (file system, rete, database) al posto del thread dell'event loop.
- **Callback**: al termine dell'operazione la callback torna all'event loop, che può completare la risposta al client.




# Installazione

**Lab**

Node.js si può installare in tre modi:

- **Installer ufficiali** disponibili sul sito web del progetto.
- **Package manager** di sistema.
- **Version manager** come `nvm`, `nvs` o `nvm4w`, comodi per sperimentare versioni diverse

e passare frequentemente dall'una all'altra.
La corretta installazione si verifica con `node --version`, che stampa la versione di Node.js in uso.

## Node.js vs browser

L'ambiente di esecuzione determina gli oggetti e le API disponibili:

- **Ambiente browser**: JavaScript manipola principalmente il DOM, con oggetti e API della piattaforma web come `window`, `document`, Cookies, LocalStorage.
- **Ambiente Node.js**: questi oggetti e API non sono disponibili, ma si possono sfruttare i moduli della standard library per l'accesso al file system e alla rete.
- In Node.js è possibile **leggere e scrivere file** sul filesystem e **ascoltare su una socket** le richieste in arrivo.

## npm

**Definition**

**npm**

**Node Package Manager**, gestore di pacchetti che fornisce i modi per scaricare e gestire le dipendenze dei progetti Node.js. È uno dei principali motivi del successo di Node.js: a settembre 2022 contava oltre 2,1 milioni di pacchetti, il più grande repository monolinguaggio al mondo, con un pacchetto per quasi tutto.

- npm è installato di default insieme a Node.js; la sua disponibilità si verifica con `npm --version`.
- Un pacchetto npm si crea nella directory corrente con `npm init`, che chiede le informazioni sul pacchetto: nome, versione, repository (se presente), licenza, autore, *entry point* (il primo file JavaScript eseguito) e comando di test (se presente).
- Per creare un pacchetto compatibile con i **moduli ES** si usa `npm init es6`.

### Il file package.json

**Definition**

**package.json**

**File descrittore del pacchetto** generato da `npm init` nella stessa directory: contiene tutte le informazioni sul pacchetto, incluse le dipendenze e i comandi per eseguirlo. È la controparte npm del `pom.xml` di Maven.

Le sue proprietà principali sono:

- **`main`**: file (es. `app.js`) eseguito quando il codice client importa il pacchetto; in genere esporta le variabili e le funzioni pubbliche.
- **`type`**: imposta il pacchetto a modulo ES con il valore `"module"`.
- **`scripts`**: task da riga di comando, eseguibili con `npm <taskname>`; il task `test` è definito di default, mentre un task `start` si aggiunge modificando la proprietà.

```JSON
{
  "name": "hello-web-technologies",
  "version": "0.0.1",
  "main": "app.js",
  "type": "module",
  "scripts": { "start": "node app.js" },
  "dependencies": { "cowsay": "^1.5.0" }
}
```

### Installare le dipendenze

Per installare una dipendenza si esegue `npm install <nomepacchetto>`, con due effetti:

- La dipendenza viene tracciata nella nuova sezione `dependencies` del `package.json`.
- I pacchetti vengono scaricati nella directory `node_modules`, comprese le loro dipendenze ricorsive.

L'importazione di una dipendenza usa la sintassi dei moduli ES:

```JavaScript
import cowsay from 'cowsay';
console.log(cowsay.think({ text: 'Hello Web Tech!' }));
```

### Distribuire un pacchetto

Per distribuire un pacchetto npm bastano il codice proprio e il descrittore `package.json`: la directory `node_modules` non va inclusa nel versioning.

Il comando `npm install`, eseguito senza argomenti, installa tutte le dipendenze elencate nel `package.json`.

## Una prima applicazione web

La standard library include il modulo `http`, con cui un server Node.js resta in ascolto su una porta e genera risposte HTML al volo:

```JavaScript
import http from 'http';
const PORT = 3000;
http.createServer(function(request, response) {
  response.writeHead(200, { 'Content-Type': 'text/html' });
  response.write('<!DOCTYPE html><html><body><h1>Hello Web Tech</h1></
    body></html>');
  response.end();
}).listen(PORT);
```

# Debugging in VS Code

Lab

### Auto Attach

L'Auto Attach si attiva e configura così:

- Il modo più semplice per fare debugging è abilitare la funzione **Auto Attach**, che collega automaticamente un debugger ai processi Node.js avviati da VS Code.
- Si attiva con `CTRL+SHIFT+P` aprendo la palette dei comandi e cercando «Toggle Auto Attach».
- La modalità **Smart** collega il debugger solo agli script eseguiti fuori da `node_modules`.

A debugging avviato si lavora così:

- I **breakpoint** si aggiungono cliccando accanto al numero di riga; quando l'esecuzione li raggiunge, il codice si mette in pausa.
- Il pannello **Run and Debug** permette di ispezionare le variabili, osservare espressioni (*watch*), consultare lo *call stack* e altro, in modo più efficace del debugging via `console.log`.

### Azioni di debug

Le **debugging actions** permettono un controllo fine del flusso di esecuzione:

| Azione | Descrizione |
|---|---|
| Resume | Riprende il normale flusso di esecuzione, fino al prossimo breakpoint |
| Step Over | Esegue l'istruzione successiva come unità singola, senza ispezionarne i passaggi interni |
| Step Into | Entra nell'istruzione successiva per seguirne l'esecuzione riga per riga |
| Step Out | Da dentro un metodo, completa il metodo corrente come un unico comando e torna al contesto precedente |
| Restart | Termina l'esecuzione e riavvia il debugging con la configurazione corrente (non funziona con Auto Attach) |
| Disconnect | Termina la sessione di debug corrente |

## Live reloading

A ogni modifica del codice l'intero server va riavviato manualmente (`npm start`, `node app.js`, …), il che rende lo sviluppo poco agevole.

- Le utility di **live reloading** monitorano il codebase e riavviano il server di sviluppo quando serve.
- **nodemon** è una di queste utility: si installa con `npm install nodemon` e si usa sostituendo `node app.js` con `nodemon app.js`.